VCFO Suite — Complete Knowledge Transfer

A-to-Z handover document for a fresh Claude Code instance (or a new developer)

Field	Value
Product	VCFO Suite — compliance cockpit for Indian professional-services firms
Owner	Prithish (SBC LLP / VCFO Suite)
Document date	2026-10-16
Describes code at	branch `client-dashboard-v7`, commit `97dabd6` + uncommitted work
Live production	`https://app.sbctrack.in` (AWS App Runner, ap-south-1)
Size of codebase	~1,135 TypeScript/TSX files, 185 test files, ~1,560 passing tests

---

Contents

0. How to use this document
1. What the product actually is (in plain English)
2. The five roles — who does what
3. The technology stack, and why each piece
4. The architecture rules you must not break
5. Getting it running from zero
6. Repository map — what lives where
7. The data model — 37 tables
8. Access control — "Path A"
9. Auth, routing and the API surface
10. The checklist engine — the heart of the product
11. Generated documents — the biggest time-saver
12. Compliance calendar and filings
13. Documents and object storage
14. Email — three paths, one dispatcher
15. WhatsApp notifications
16. In-app notifications and announcements
17. Ask VCFO — the in-app AI assistant
18. The design system
19. Background jobs (Inngest)
20. Testing and verification
21. Deployment
22. Environment variable reference
23. The owner and how they work
24. The gotchas index
25. Current state — what is done, what is uncommitted, what is deferred
26. How to continue — the onboarding sequence for a new instance
Appendix A — Command cheat sheet
Appendix B — Glossary (Indian incorporation and compliance)
Appendix C — Document provenance

0. How to use this document

Read this document **top to bottom once** before you touch any code. It is written so that a person (or an AI agent) who has never seen this repository can understand *what the product is*, *why it is built this way*, *where everything lives*, and *what the rules are* — and then safely continue the work.

The document has three kinds of content, and you should treat each differently:

1. **Explanation** (sections 1–4) — what the business does and what the software is for. Read it to build a mental model. You cannot make good decisions in this codebase without understanding Indian company incorporation, because the data model mirrors it exactly.
2. **Reference** (sections 5–18) — the architecture, the tables, the routes, the rules. Come back to these while working.
3. **Operating instructions** (sections 19–26) — how to run it, how to verify it, the owner's working preferences, and the hard-won gotchas. **Section 23 (owner preferences) and section 24 (gotchas) are the two sections that will save you the most time.** Many of the lines in section 24 represent a bug that cost hours to find. Do not re-discover them.

The three documents that are the live source of truth

This handover PDF is a snapshot. Inside the repository, three things are kept continuously up to date and **they win over this document if they disagree**:

File	What it holds
`CLAUDE.md`	The build instructions and the golden rules. Read first, every session.
`docs/context/STATE.md`	What is actually verified green right now, feature by feature, with dates.
`docs/context/HANDOFF.md`	The latest session's work, the owner's preferences, and the next steps.
`docs/context/notes/*.md`	Twelve per-area files of gotchas "already paid for once".

Your first action in a new session is to read those four, in that order. Then keep them updated as you work — that is how this project maintains memory across sessions and across different AI instances.

A note on honesty in this document

Where something is incomplete, deferred, broken, or uncertain, this document says so plainly. A handover that hides rough edges produces a successor who breaks things. Look for the phrases **"not done"**, **"deferred"**, **"uncommitted"**, and **"known bug"**.

---

1. What the product actually is (in plain English)

1.1 The business problem

SBC is an Indian chartered-accountancy / professional-services firm. A large part of its work is **helping foreign companies set up a legal entity in India** — typically a Global Capability Centre (GCC), which is a subsidiary that a foreign parent opens in India to run engineering, finance, or support operations.

Setting up that entity is not one task. It is a **long, strictly ordered sequence of about 48 steps** spread across four phases, each with its own forms, its own documents, its own government portal, and its own statutory deadline. It takes months. It involves:

- the **MCA** (Ministry of Corporate Affairs) — company registration
- the **Income Tax Department** — PAN, TAN
- **GST** authorities

- **RBI / FEMA** — foreign-investment reporting
- **EPFO / ESIC** — employee provident fund and insurance
- **Customs, DGFT, Shops & Establishments, Professional Tax, trademark offices** — depending on what the company will do

Before this software existed, the firm ran this on **spreadsheets, email threads, and WhatsApp**. The problems that caused:

- Nobody could answer "where exactly is my company right now?" without asking a person.
- Documents arrived by email and were re-requested because nobody could find them.
- Deadlines were tracked in one person's head.
- The client had no visibility, so they phoned constantly.
- When a staff member left, their context left with them.

1.2 What the software does

VCFO Suite is a **web application** that turns that process into a single shared, auditable workflow. In one sentence:

> It walks a client and the firm's staff through the ~48-step incorporation checklist together — collecting answers and documents, generating the legal documents from those answers, enforcing who may do what and when, notifying everyone by email and WhatsApp, and then rolling the finished company onto a recurring statutory compliance calendar.

Concretely, it gives you:

1. **A gated checklist per company.** Step 7 cannot open until step 6 is genuinely finished. Each step is a form (text, dates, dropdowns, repeating lists of directors, file uploads) plus validation rules specific to that step.
2. **Document generation.** From the answers already collected, the app generates the actual legal `.docx` documents — board resolutions, MOA, AOA, DIR-2 consent, DIR-8 declaration, INC-9, NOC letters, subscription sheets, per-director declarations — with the right names, addresses, share counts and jurisdictions filled in. This is the single biggest time-saver in the product.
3. **A client portal.** The client logs in and sees exactly what is needed from them, what the firm is doing, who holds the ball, and what is coming up — with nothing they are not supposed to see yet.
4. **Role-based workflow with approvals.** A junior staff member ("Project Lead") prepares work; a Project Manager approves it; only then does the client see it. There are deliberate exceptions, documented in section 10.
5. **A compliance calendar.** Once the company is incorporated (certificate of incorporation issued), it gets recurring filing obligations — GST returns, TDS, annual returns, FEMA filings — generated on a schedule with due dates and status.
6. **Notifications.** Every meaningful event emails the right people and (when enabled) sends a WhatsApp nudge. Email is the system of record; WhatsApp is only a nudge.
7. **An in-app AI assistant ("Ask VCFO")** that answers client questions from firm-reviewed content plus the client's own live project state.
8. **A full audit trail**, visible to staff and (scoped) to the client.

1.3 The four phases of the incorporation journey

This is the spine of the whole product. Memorise it.

Phase	Name	Steps	What happens
Phase 1	Name Application (SPICe+ Part A)	5 (`pre-1` … `pre-5`)	Collect company and parent-entity details, get a board resolution from the parent authorising the Indian entity, apply to MCA to reserve the company name, receive name approval.

Phase	Name	Steps	What happens
Phase 2	Incorporation (SPICe+ Part B)	9 (`pre-13` , `pre-14` , `pre-15` , `pre-16` , `pre-7` , `pre-9` , `pre-10` , `pre-11` , `pre-12`)	Capital structure, registered office, proposed directors (a repeating list, each with KYC), subscribers, KYC + digital signature certificates, document execution, MCA filing, query resolution, **Certificate of Incorporation (COI)**.
Phase 3	Post-Incorporation	11 (`post-1` … `post-11`)	Everything that must happen in the first weeks of the company's life: bank account, share capital infusion, first board meeting, statutory registers, auditor appointment, FEMA/FDI reporting.
Phase 4	Registration	23 (`reg-1` … `reg-24` , minus legacy `reg-2`)	All the operating registrations: GST, PAN/TAN, Shops & Establishments, Professional Tax, EPFO, ESIC, Import-Export Code, Customs, trademark, LEI, and so on.

Four phases, **48 active steps**. The step ids are not sequential by number — the *order* comes from the `itemIds` array of each phase group in `src/data/checklist.ts` , not from the numbers in the ids. This matters: `pre-13` comes **before** `pre-7` in Phase 2 because the firm restructured Part B in September 2026 and kept the old ids to avoid migrating stored data.

Two steps, `pre-6` (Director KYC) and `pre-8` (Document Execution), plus `reg-2` , still exist in the catalog but belong to **no phase**. They are *legacy rows*: old engagements have data stored under them, and some code still reads that data as a fallback. **Do not delete them.**

1.4 "Dependent" vs "Independent" companies

A company being incorporated is one of two kinds, and this changes the checklist:

- **Dependent** (`ownership_type = 'subsidiary'` , the default) — there is a **foreign or Indian parent entity**. The parent must pass a board resolution authorising the Indian subsidiary, so steps `pre-2` and `pre-3` (parent board resolution and its signed copy) apply, and `pre-1` asks for the parent's details, country, state, proof documents and authorised signatory.
- **Independent** (`ownership_type = 'independent'`) — no parent. The parent sections of `pre-1` are hidden, `pre-2` and `pre-3` are seeded as *not-applicable* at creation, FEMA/transfer-pricing calendar rows never apply, and the board-resolution and authorisation documents are dropped from the document pack.

The helper that decides this is `fieldsForOwnership` in `src/lib/checklist-responses.ts` , which delegates to `src/lib/part-a-sections.ts` . **Never hard-code a tab list or field list for `pre-1` .**

---

2. The five roles — who does what

Everything in this app is shaped by five roles. The code role names differ from the display names, which is a frequent source of confusion.

Code role (DB + session)	Display name in the UI	Home page	What they can do
`super_admin`	**Super Admin**	`/app/super/dashboard`	Bird's-eye over the whole firm. May enter **every** role's workspace for review (the only role allowed to cross segments). Can preview the client portal as a client.
`admin`	**Firm Admin**	`/app/admin/dashboard`	Firm-wide access to all engagements. Creates projects, manages people, approves managers' change requests, deletes/restores projects.
`manager`	**Project Manager**	`/app/manager/dashboard`	Scoped to engagements where `manager_id = their user id` (plus `engagement_managers` membership). **Approves the Project Lead's work**, which is what releases it to the client. Must file a change request for three high-risk actions.

Code role (DB + session)	Display name in the UI	Home page	What they can do
`intern`	**Project Lead**	`/app/intern/today`	⚠️ The code name is `intern` but the product name is **Project Lead**. Scoped to assigned engagements. Does the actual preparation work: fills steps, generates documents, uploads filings.
`client`	**Client**	`/app/client/overview`	Scoped to their own engagement(s). Fills the steps that are theirs, uploads documents, approves/requests changes, sees progress and the compliance calendar.

2.1 The naming trap

`intern` in the code means "Project Lead" in the product. Variables, tables, API routes and URL segments all say `intern` (`/app/intern/today`, `profiles.intern_id`, `engagements.intern_id`, `useInternPortfolio`). The UI never says "intern" — it says "Project Lead". Do not rename the code role; too much depends on it. Just be aware.

2.2 The approval chain (and its exceptions)

The default chain is:

```
Project Lead fills a step
      |  (a plain Save is a PRIVATE DRAFT — the manager sees nothing)
      ▼
Project Lead clicks "Request manager approval" / "Submit"
      |  (step now reads "waiting"; still NOT a green tick)
      ▼
Project Manager clicks "Accept"
      |  (this is the delivery — it stamps deliveredToClientAt)
      ▼
Client can finally see the step
```

There are **three deliberate bypasses** of the manager, and you must not "fix" them:

1. **Pre-incorporation direct flow (Oct 2026)** — for every active pre-incorporation step *except* `pre-2`, the client and the Project Lead fill the same form and either of them can submit it with **no approval at all**. See section 10.6. This was a deliberate product decision to remove friction.
2. **Incorporation-draft sharing (`pre-7`)** — the Lead can share generated drafts straight to the client for download.
3. **Board-resolution finalize (`pre-2`)** — this is the one remaining approval in pre-incorporation, and it is manager-only.

---

3. The technology stack, and why each piece



Layer	Choice	Why this choice
Framework	**Next.js 16** (App Router), React 19	One deployable unit: UI, API routes and server components in one repo. No separate backend to keep in sync.
Language	**TypeScript 6**	`strict: false`, `strictNullChecks: false` — a pragmatic setting inherited from the original codebase. Do not turn strict on casually; it would surface hundreds of errors.
Database	**Postgres 16** + **Drizzle ORM**	Drizzle gives typed SQL without a heavy runtime. Schema lives in `src/db/schema.ts`; migrations are real SQL files in `src/db/migrations/`.
Auth	**Auth.js (NextAuth v5 beta)** with credentials + bcrypt	No external identity provider to pay for or depend on. Sessions are JWT; role/scoping ids are **re-read from the `profiles` table on every request** so a stale JWT cannot grant or block access.
Object storage	**S3 API** — MinIO locally, real S3 on AWS	Dev/prod parity: the same `src/storage/s3.ts` code talks to both. Only the endpoint env var changes.
Background jobs	**Inngest**	Durable, retryable steps with throttling and dedupe ids, without standing up a queue. Used for the compliance generator, WhatsApp sends, RSS ingest, and Ask VCFO ingestion.
Email	**Resend** (default) or **AWS SES** behind one dispatcher, plus **Microsoft Graph** for staff-composed mail	Resend for transactional system mail; Graph so a Project Lead's email to a client genuinely comes *from that person's Outlook mailbox*.
WhatsApp	**AWS End User Messaging (Social)** only	Chosen so the Meta per-message fee lands on the **same AWS invoice** as SES/RDS/S3. A Twilio transport was tried and deliberately removed.
Docx generation	**docxtemplater** + **pizzip**	Templates are real `.docx` files with tags; the app fills them. Preview in-browser with `docx-preview`.
PDF generation	**@react-pdf/renderer**	Client report PDF, Ask VCFO monthly brief.
AI	**Anthropic SDK** (direct or via **Bedrock**)	Ask VCFO: Haiku as a cheap guard, Sonnet for answers.

Layer	Choice	Why this choice
UI	**Tailwind 3.4** + **shadcn/ui** (Radix) + **Framer Motion** + **Recharts**	Fast, token-driven, accessible primitives.
Testing	**Vitest** (unit/integration, ~1,560 tests) + **Playwright** (e2e) + **PGlite** (in-memory Postgres for repository tests)	
Hosting	**AWS App Runner** + **RDS Postgres** + **S3** + Secrets Manager, Terraform in `infra/`	Smallest architecture that fits. Explicitly **not** Kubernetes, not Lambda-per-route, not Cognito.

3.1 Important stack warnings

- **Next.js 16 is not the Next.js most training data knows.** `CLAUDE.md` contains a standing instruction: read the relevant guide in `node_modules/next/dist/docs/` before writing routing or server-component code. APIs and conventions differ.
- **Tailwind 3.4**, not 4. Opacity utilities only come in steps of 5 (`bg-primary/25`, not `/23`).
- **`pg` ≥ 8.16 treats `sslmode=require` as verify-full.** Connecting to RDS therefore needs `NODE_EXTRA_CA_CERTS=certs/rds-global-bundle.pem`. An explicit `ssl: { rejectUnauthorized: false }` does **not** override it.
- **`npm install` needs `legacy-peer-deps=true`** (already in `.npmrc`) because of an `eslint@10` vs `eslint-plugin-jsx-a11y` peer conflict.
- **PowerShell has no `&&`.** On Windows, chain with `;` or `; if ($?) { … }`.

---

4. The architecture rules you must not break

These five rules are from `CLAUDE.md` and they are the reason this codebase is still maintainable at 1,100+ files. Violating them is the fastest way to create a security hole or an unmaintainable tangle.

Rule 1 — The seam is sacred

> **Only `src/db/repositories/*` may import `db`.**

Views, domain logic, components and API routes call repository functions. They never write SQL and never import the Drizzle client. If you are tempted to `import { db }` in a component or a route — stop, and add a repository function instead.

Why this matters: the seam is what made the move from Supabase to Postgres-on-AWS a config change instead of a rewrite, and it is where **all** access control lives (Rule 2). A single bypass puts an unscoped query into production.

The only exceptions are **system jobs** (Inngest), which may touch `db` without an `AuthContext` — e.g. `systemGenerateComplianceInstances` in `compliance.ts`, and the feed-ingest helpers in `announcements.ts`. Those functions are named `system*` precisely so the exception is visible. Routes still must not import `db`.

Rule 2 — Every repository takes an `AuthContext` and scopes by role

```
export interface AuthContext {
  userId: string;
  email: string;
  name: string;
  role: 'super_admin' | 'admin' | 'manager' | 'intern' | 'client';
  internId?: string;   // scoping key for Project Leads
  clientId?: string;   // scoping key for clients
}
```

Every repository function takes this as its **first argument** and filters by the role's scoping keys. This approach is called **"Path A"**: access control is enforced **in the repository layer, not in Postgres RLS**. The original Supabase app used RLS policies; those policies are now the **specification**, reproduced as a table in `src/db/repositories/README.md`. See section 8.

`engagements.ts` is the reference implementation — copy its shape.

Rule 3 — Don't touch the LIFTed domain code unless a test fails

About 80% of this codebase was **lifted unchanged** from the original SBC-Track application: the checklist catalog, every per-step validation module, docx generation, compliance math, and most UI components. It was already tested in the original. If it is not failing, leave it alone.

Rule 4 — Dev/prod parity

Local uses Docker Postgres + MinIO; AWS uses RDS + S3. **Never hard-code an endpoint.** Everything flows through environment variables, resolved in `src/db/client.ts`, `src/storage/s3.ts` and `src/lib/site-url.ts`.

Rule 5 — Verify each phase before stacking more work

```
npm run typecheck && npm run test
```

Both must be green before you move on. Do not stack unverified work. The current verified gate is recorded in `docs/context/HANDOFF.md` with a date — update it when you re-run.

Things explicitly NOT to do

- Do **not** reintroduce Supabase.
- Do **not** add Kubernetes, microservices, or Lambda-per-route. App Runner + RDS + S3 is the architecture.
- Do **not** put secrets in the client bundle; do not commit `.env.local`.
- Do **not** bypass the repository seam "just for this one query".
- Do **not** add keep-alive "registry" files that import unused modules to silence dead-code tooling — that trick previously hid ~60 dead files.
- Do **not** reintroduce a WhatsApp "provider" field. There is one transport.

---

5. Getting it running from zero

5.1 Prerequisites

- **Node.js 20+** (the owner's machine runs v24.19.0)

- **Docker Desktop** — this runs local Postgres and MinIO. **It must be running before `npm run dev`.** If it is not, login fails with "We couldn't sign you in" and the server log shows `ECONNREFUSED`. That symptom has wasted time more than once.
- Windows 11 in the owner's case. The repo works on macOS/Linux too.

5.2 First-time setup

```
cp .env.example .env.local
# then set AUTH_SECRET in .env.local:
#   openssl rand -base64 32

npm install                 # needs legacy-peer-deps (already in .npmrc)
npm run infra:up            # start Postgres + MinIO in Docker (idempotent)
npm run db:migrate          # apply the 25 migrations to local Postgres
npm run db:seed             # demo super/admin/manager/lead/client + engagement
npm run dev                 # http://localhost:3000
```

Optional extras:

```
npm run db:seed-demo        # a richer demo firm
npm run db:purge-demo       # remove demo data
npm run compliance:generate # fill the compliance calendar by hand
npm run inngest:dev         # Inngest dev server on :8288 (needed for WhatsApp)
```

5.3 Demo logins

Created by `npm run db:seed`. **These passwords are weak and must be changed or purged before any real user touches a deployment.** That item is still on the deferred list for RDS.

Role	Email	Password
Super Admin	`super@vcfo.local`	`super123`
Firm Admin	`admin@vcfo.local`	`admin123`
Project Manager	`manager@vcfo.local`	`manager123`
Project Lead	`intern@vcfo.local`	`intern123`
Client	`client@vcfo.local`	`client123`

Note: `admin@vcfo.local` was absent from the owner's local DB as of 2026-09-07 — use `admin-nadia@demo.vcfo.local` / `demo1234` from `db:seed-demo` instead.

MinIO console (to see uploaded files): `http://localhost:9001`, credentials `vcfo_minio` / `vcfo_minio_local_dev`.

5.4 Useful local facts

- The Postgres user is **`vcfo`, not `postgres`**: `docker exec vcfo-postgres psql -U vcfo -d vcfo`. Connecting as `postgres` fails with `role "postgres" does not exist`.
- Containers are named `vcfo-postgres` and `vcfo-minio`.
- The database volume is `vcfo-pgdata`. **That volume is the database** — back it up if real client data is in a local pilot.
- MinIO's public images were pulled in Sept 2026; `docker-compose.yml` uses **`pgsty/silo:latest`**, a drop-in fork with the same S3 API, the same `MINIO_*` env vars and the same on-disk format.

5.5 Running it as an office server (LAN pilot)

```
npm run dev:lan            # next dev bound to 0.0.0.0:3000 (hot reload)
# or, production-like:
npm run build && npm run start:lan
```

Then testers open `http://<server-LAN-IP>:3000`.

Two traps here:

1. **Leave `AUTH_URL` unset** (`AUTH_TRUST_HOST=true` is already on). Auth.js's `reqWithEnvURL` rewrites the request origin to `AUTH_URL` when it is set, so pinning `http://localhost:3000` — or a previous Cloudflare tunnel hostname — breaks login on the host you actually opened. Same for `NEXT_PUBLIC_SITE_URL`.
2. **Next 16 blocks cross-origin `/_next/*`** (JS chunks and HMR, not HTML or API) unless the browser's Origin is allow-listed in `next.config.mjs`. The symptom is nasty: *the page loads but the login fields and toggles are dead.* LAN ranges and `*.trycloudflare.com` are already listed. For other hosts set `DEV_TUNNEL_HOST` or `ALLOWED_DEV_ORIGINS` and **restart** — `next.config` is read at boot. Verify in DevTools then Network that `/_next/static/chunks/*` returns **200**, not **403**.

5.6 Verification commands (the "gate")

```
npm run typecheck                          # tsc --noEmit
npm run test                               # vitest run
npx vitest run --testTimeout=60000         # needed: docx/zip/Ask tests exceed
                                           # the 5s default while next dev runs
npm run lint
npx knip --reporter compact                # dead-code scan
```

On Windows PowerShell: `npx vitest run <files>` and `npx tsc --noEmit -p .`.

`tsc` is **incremental** here (`tsconfig.tsbuildinfo`). For a gate you fully trust, use `--incremental false` — a stale build-info file once hid a real test typing error.

`knip`'s only expected residue is `src/test/server-only-stub.ts` (a Vitest alias, not an import). Anything else it reports is genuinely new dead code.

---

6. Repository map — what lives where

```
vcfo_suite/
├── CLAUDE.md                  <- build instructions + golden rules (read first)
├── README.md                  <- how to run it, incl. the LAN pilot
├── docker-compose.yml         <- local Postgres + MinIO (+ bucket bootstrap)
├── drizzle.config.ts          <- Drizzle Kit config (schema -> migrations)
├── proxy.ts                   <- Next middleware: auth + role-segment guard
├── next.config.mjs            <- incl. allowedDevOrigins for LAN/tunnel access
├── Dockerfile                 <- multi-stage, standalone output, RDS CA baked in
├── buildspec.yml              <- CodeBuild spec for the AWS image build
├── certs/rds-global-bundle.pem   Amazon RDS CA bundle (NODE_EXTRA_CA_CERTS)
|
├── app/                       <- Next.js App Router
|   ├── page.tsx               <- marketing landing
|   ├── login/                 <- Auth.js sign-in
|   ├── invite/[token]/        <- invited-user acceptance
|   ├── contact/, roles/       <- public pages
|   ├── globals.css            <- design tokens + every utility class
|   ├── providers.tsx          <- TanStack Query, LazyMotion, theme, AppContext
|   ├── app/                   <- THE PRODUCT (authenticated)
|   |   ├── super/   (6 pages)
|   |   ├── admin/   (30 pages)
|   |   ├── manager/ (26 pages)
|   |   ├── intern/  (21 pages)   <- "Project Lead"
|   |   └── client/  (18 pages)
|   └── api/                   <- ~120 route handlers (see section 9.4)
|
├── src/
|   ├── data/                  <- PURE domain data (no DB, no React)
|   |   ├── checklist.ts       <- * THE 48-STEP CATALOG + phase groups
|   |   ├── compliance.ts      <- compliance filing types
|   |   ├── engagements.ts     <- engagement domain types
|   |   ├── nic-2008.json      <- 1,302 NIC business-activity sub-classes
|   |   ├── statutory-calendar-fy2627.ts   fixed FY master deadline calendar
|   |   └── ask/               <- Ask VCFO content: topics, glossary, playbook
|   |
|   ├── db/                    <- THE SEAM
|   |   ├── client.ts          <- the Drizzle/pg client (nothing else imports pg)
|   |   ├── schema.ts          <- 37 tables + 4 enums, 1,042 lines
|   |   ├── migrations/        <- 25 SQL migrations + Drizzle meta snapshots
|   |   └── repositories/      <- 40+ repository modules + README (the RLS spec)
|   |
|   ├── auth/
|   |   ├── config.ts          <- Auth.js full config (providers, bcrypt, pg)
|   |   ├── config.base.ts     <- shared config
|   |   ├── edge.ts            <- * edge-safe wrapper; proxy.ts MUST use this
|   |   └── guards.ts          <- requireAuth / requireRole / requireAnyRole
|   ├── storage/
|   |   ├── s3.ts              <- S3/MinIO put/get/signed-url
|   |   └── board-resolution.ts
```

```
│   ├── jobs/                 <- Inngest functions
│   │   ├── client.ts         <- the Inngest client
│   │   ├── compliance-generate.ts
│   │   ├── whatsapp-send.ts
│   │   ├── announcement-feeds.ts
│   │   └── ask-ingest.ts
│   │
│   ├── lib/                  <- ~200 pure modules + their colocated tests
│   │   ├── checklist-*.ts    <- * the checklist engine (see section 10)
│   │   ├── board-resolution*.ts
│   │   ├── incorporation-docs/   per-director document generation
│   │   ├── doc-pack/         <- the read-only document pack model
│   │   ├── noc/              <- NOC letter variants
│   │   ├── compliance/       <- incorporation-state, scope
│   │   ├── email/            <- dispatcher, transports, process fan-out
│   │   ├── notify/           <- WhatsApp dispatcher, EUM transport, SNS verify
│   │   ├── outlook/          <- Microsoft Graph
│   │   ├── ask/              <- Ask VCFO pipeline, tools, provider, ingest
│   │   ├── orbit/            <- * NEW UI: orbit dashboards (active work)
│   │   ├── api/              <- route helpers (parseJsonBody, doc-pack guard)
│   │   └── pre-inc-direct-flow.ts   * the one policy module for direct flow
│   │
│   ├── components/           <- ~40 folders of UI
│   │   ├── shell/            <- AppShell, RoleSidebar, TopBar, CommandPalette,
│   │   │                        NotificationsBell, crumbs, PageBackButton
│   │   ├── dash/             <- DashHero, DashSection, DashDataTable, charts
│   │   ├── client/           <- the client portal components
│   │   │   └── orbit/        <- * NEW UI: orbit components (active work)
│   │   ├── intern/           <- Lead dashboard = the canonical design system
│   │   ├── incorporation/    <- the checklist form engine UI
│   │   ├── board-resolution/, doc-pack/, docx-preview/
│   │   ├── compliances/, schedule/, notifications/, announcements/
│   │   ├── ask/              <- the Ask VCFO panel
│   │   ├── ui/               <- shadcn primitives
│   │   └── noir/             <- house primitives (Surface, ProgressRing, ...)
│   │
│   ├── views/                <- * THE REAL PAGE UI lives here, not in app/
│   │   └── client/, intern/, admin/, super/, compliances/, incorporation/,
│   │       engagement/, knowledge-bank/, vault/, settings/, auth/,
│   │       notifications/, announcements/, staff/
│   ├── context/              <- AppContext + use-app-provider-value
│   ├── hooks/                <- React hooks (incl. hooks/ask)
│   └── test/                 <- test setup, PGlite harness, server-only stub
│
├── scripts/                  <- seed, compliance generate, docx tag builders,
│                                Ask topic report/publish, purge demo
├── tests/                    <- Ask VCFO evals and shared test assets
├── e2e/                      <- Playwright specs
├── infra/                    <- Terraform for AWS (10 .tf files + README)
├── tools/
└── docs/
    ├── KNOWLEDGE-TRANSFER.md  <- this document
```

```
    ├── context/              <- * LIVE WORKING MEMORY
    │   ├── STATE.md          <- verified state, feature by feature
    │   ├── HANDOFF.md        <- latest session, owner preferences, next steps
    │   ├── AWS-DEPLOY.md     <- the deploy checklist
    │   └── notes/            <- 12 files of gotchas (see section 24)
    └── specs/                <- specs for in-progress features
        ├── ask-vcfo/         <- context, status, recon, eval, topic review
        ├── assist-profile/
        ├── doc-gaps/
        └── pre-inc-direct-flow/   CONTEXT, AUDIT, REPORT
```

6.1 The single most important structural fact

> **`app/app/{role}/.../page.tsx` files are ROUTE SHELLS ONLY.**

A typical page file is literally:

```
export default function Page() {
  return <InternToday />;
}
```

The **real UI lives in `src/views/`** and `src/components/`. When you are told "the client home page", that is `src/views/client/Overview.tsx`, not `app/app/client/overview/page.tsx`. Searching only under `app/` will make you believe the app is empty.

---

7. The data model — 37 tables

All tables are defined in `src/db/schema.ts` (1,042 lines). Migrations are real SQL in `src/db/migrations/0000` through `0024`. Drizzle Kit generates them from the schema (`npm run db:generate`); you apply them with `npm run db:migrate`.

7.1 Enums

Enum	Values
`trak_role`	`super_admin`, `admin`, `manager`, `intern`, `client`
`engagement_stage`	the lifecycle stage of a project (Pre-Incorporation onward)
`engagement_health`	health indicator for dashboards
`whatsapp_status`	`queued`, `sent`, `delivered`, `read`, `failed`, `skipped`

7.2 Core identity and project tables

Table	Purpose and key columns
`profiles`	Every user. `role`, `email`, `name`, `status`, `intern_id` and `client_id` (the **scoping keys** that replaced the old RLS `my_intern_id()` / `my_client_id()` functions), `reports_to_manager_id`, `phone_e164`, `whatsapp_opt_in_at`, `avatar_object_key`. Passwords are bcrypt hashes.
`engagements`	The central table. One row = one company being incorporated ("a project"). Holds `slug`, company name, `stage`, `health`, `ownership_type` (`subsidiary` / `independent`), parent-entity fields, subsidiary fields, `incorporation_date`, `manager_id`, `intern_id`, `client_user_id`, `client_id`, `admin_id` (legacy), `deleted_at` (**soft delete**), `schedule` (jsonb — manager date windows), and **`checklist_state` (jsonb)**.

Table	Purpose and key columns
`engagement_clients`	Multi-client membership (`owner` / `member`). The primary client is still on `engagements.client_user_id` for legacy reasons.
`engagement_leads`	Additional Project Leads beyond `engagements.intern_id` .
`engagement_managers`	Additional / co-managers beyond `engagements.manager_id` .
`engagement_change_requests`	A manager's request for a high-risk action, with `payload` (proposed values) and `preview` (frozen before/after). Approving one **applies** it.

`engagements.checklist_state` — the most important column in the app

The entire 48-step checklist for a company is **one jsonb blob** on the engagement row, keyed by step id:

```
{
  "pre-1": {
    "status": "completed",
    "responses": { "companyName": "Acme India Pvt Ltd", "nicCode": "62011" },
    "reviewStatus": "accepted",
    "deliveredToClientAt": "2026-09-12T...",
    "clientSubmittedAt": "...",
    "submittedByName": "...", "submittedByRole": "client",
    "lastEditedAt": "...", "lastEditedByName": "...",
    "approval": { },
    "clientFillRequest": { },
    "notes": "staff-only"
  },
  "pre-13": { }
}
```

Consequences you must know:

- **It is expensive to read.** `listEngagements` deliberately does **not** select `checklist_state` . Dashboards hydrate status from `GET /api/checklist-index` , which strips answers **in SQL**. Never put a `refetchInterval` on a list that includes the blob.
- **Anything stored at the top level would be read as a step.** That is why `schedule` is its own column — `normalizeEngagementChecklistState` would otherwise treat a `schedule` key as a step.
- **Writes merge onto the persisted row only.** `patchChecklistItem` accepts the browser's copy of `current` and **ignores it**, because that copy has been redacted for the caller's role and writing it back would wipe another party's data.
- **Repeating lists live inside `responses`** , not in a new table. See section 10.4.

7.3 Document tables

Table	Purpose
`documents`	The document index. `shared_with_client` controls client visibility. Index rows for files on *unreleased* steps are dropped at read time.
`document_templates`	Templates for generation.
`document_requests`	"Please send us X" requests to the client.

Table	Purpose
`engagement_board_resolutions`	The `pre-2` board resolution: draft content, status (`draft` / `pending` / `finalized`), file keys.
`knowledge_bank_folders`	Nested firm reference-library folders (`parent_id` null = root).
`knowledge_bank_files`	Files in those folders (`folder_id` null = root).

Checklist file uploads are **not** a table: the bytes go to S3 under `milestone-documents/`, the path is stored in the step's `responses`, and staff uploads additionally get a `documents` index row so the vault is complete.

7.4 Compliance tables

Table	Purpose
`compliance_obligations`	The master catalog, upserted from `COMPLIANCE_OBLIGATIONS` on the generator's first run.
`engagement_compliance_triggers`	Which obligations apply to which company.
`compliance_instances`	The generated, dated filings — one row per obligation per period. Has `window_*` columns (migration 0019) for manager-set date windows.
`compliance_events`	Status changes on an instance.

7.5 Communication tables

Table	Purpose
`email_templates`	Firm-scoped compose library. `branding` = `sbc` or `plain`.
`email_template_bindings`	Template-to-event bindings.
`email_send_log`	Record of sends.
`outlook_connections`	A staff member's linked Microsoft mailbox. Tokens **encrypted with** `AUTH_SECRET`. Not an Auth.js login provider.
`notifications`	The per-user in-app bell. `user_id` scoped. `dismissed_at` hides a row from the inbox without deleting it. Extras (`kind`, `href`) are **JSON-encoded inside the** `description` **column** — a historical compromise.
`notification_deliveries`	WhatsApp delivery log: `channel`, `status`, `skip_reason`, `error_code`, `template_sid` (holds the Meta **template name**), `variables` jsonb and `retry_of` (migration 0024). **Email sends are not written here**, so the Send log's Email filter is empty.
`announcements`	Firm-wide news (Finance Act, circulars). Not the bell. `kind` + `author_role`.
`announcement_sources`	Allow-listed official RSS/Atom feed URLs.

7.6 Workflow tables (these replaced localStorage)

Table	Purpose
`tasks`	Staff tasks. **Also hosts personal todos**: `engagement_id` null, `assigned_to` = owner, `step_id` prefixed `todo:`, served by `/api/todos` and excluded from `/api/tasks`.
`activity`	Activity feed rows.
`invites`	Pending user invitations.
`audit_events`	The audit trail. `actor_user_id` is **forced to the session user** on insert.

7.7 Ask VCFO tables

Table	Purpose
`ask_conversations`, `ask_messages`	Chat history with token usage.
`ask_documents`, `ask_chunks`	Ingested firm documents, chunked, searched with **Postgres full-text search — no pgvector**.
`client_library_items`	Answers the client chose to save.
`ask_client_prefs`	Per-client Ask preferences (migration 0023).

7.8 Migration history (what each one added)

#	Name	What it did
0000	`smiling_johnny_storm`	The initial schema.
0001	`four_role_admin`	Added the `admin` role (three roles to four).
0002-0003		Misc; nullable `intern_id`.
0004-0005	`engagement_leads`, `engagement_managers`	Multi-lead and multi-manager membership.
0006	`subsidiary_entity`	Subsidiary entity fields.
0007	`outlook_connections`	Microsoft Graph mailbox links.
0008	`email_template_branding`	`branding` column on templates.
0009-0010	`announcements`, `announcement_kind`	Firm news board.
0011	`profile_avatar`	`avatar_object_key`.
0012	`notification_dismissed_at`	Clear-without-delete on the bell.
0013	`knowledge_bank_folders`	Nested folders.
0014	`boring_nekra`	`engagement_change_requests`. **Hand-trimmed** — Drizzle emitted a large drift backlog for tables that already existed; only the new table was kept, and the snapshot is intact so future diffs are correct.
0015-0017		Misc.
0018	`yielding_young_avengers`	`ownership_type` (Dependent / Independent).
0019	`goofy_titanium_man`	`engagements.schedule` + `compliance_instances.window_*`.
0020	`boring_scrambler`	Misc.
0021	`lonely_sunfire`	Ask VCFO tables + full-text search.
0022	`rename_assist_to_ask`	Renamed "Assist" to "Ask VCFO".
0023	`ask_client_prefs`	Ask daily-nudge preferences.
0024	`whatsapp_delivery_variables`	`variables` jsonb + `retry_of` on `notification_deliveries`. **Applied locally, NOT yet on RDS.**

Migration 0024 is not applied to production RDS. The WhatsApp retry feature needs it before it will work in production.

---

8. Access control — "Path A"

8.1 The concept

The original app used **Postgres Row Level Security**. This app enforces the same rules **in the repository layer** instead. The old RLS SQL is therefore the **specification**, and it is transcribed into a table in `src/db/repositories/README.md`.

That README carries a warning you should take literally:

> Two rows in the first draft of this table were WRONG and have been corrected — **the SQL is the spec, not this table**, so re-check the migration before trusting a row.

8.2 The scoping rules, table by table

Table	Admin	Manager	Project Lead (`intern`)	Client
`engagements`	all	`manager_id = ctx.userId` (legacy fallback: `manager_id IS NULL AND admin_id = ctx.userId`)	`intern_id = ctx.internId` plus `engagement_leads`	`client_user_id = ctx.userId OR client_id = ctx.clientId` plus `engagement_clients`
`engagement_board_resolutions`	all	via owned engagement	read + insert via assigned engagement; update **only while** `status = 'draft'`	read via own engagement
`knowledge_bank_files`	all	all	**read all** + insert own (`uploaded_by = self`); cannot delete	**none at all**
`knowledge_bank_folders`	all	all	read all + insert own; **no delete**	**none**
`compliance_obligations`	all	all	read	read
`compliance_instances` / `_events` / `_triggers`	all	via owned engagement	write via assigned engagement	**read-only** via own engagement
`audit_events`	firm-wide read	only owned / assigned engagement ids — *not* own actor rows on someone else's client, and not unscoped admin noise	own actor rows **always**, plus events on assigned engagements	own engagements
`email_send_log`	read	read	none	none
`documents`	all	via owned engagements	assigned + membership only	own engagement **and** `shared_with_client = true` **only**
`notifications`	own rows (admin / manager may *create* for another user)	same	same	same
`outlook_connections`	own mailbox	own mailbox	own mailbox	**none**

Table	Admin	Manager	Project Lead (`intern`)	Client
`announcements`	read all; post + delete	read all; post + delete	read only	read only
`tasks` (personal todos)	firm-wide staff	self + leads / co-managers on scoped engagements + intern reports	**own rows only**	**none**

Mutating a personal todo is **owner-only** regardless of role.

8.3 Corrections already made (do not re-break them)

- `knowledge_bank_files` has **no** `visible_to_roles` column. Leads read every file and insert their own; **clients get nothing**.
- `knowledge_bank_folders` delete **refuses non-empty folders** — FKs are `ON DELETE restrict`, there is no cascade. Empty it first.
- `audit_events` original RLS was manager-read-only with interns seeing nothing. A **product override** gives the Project Lead their own actor rows plus events on assigned engagements — that is the shared-client trail the firm needed.

8.4 Required practice for any new repository

1. Take `AuthContext` as the **first** argument.
2. Branch on `ctx.role` and filter by the scoping keys.
3. **Add a test asserting that cross-tenant access returns nothing.**

Cross-tenant repository tests are currently on the **deferred** list — the owner has been testing manually. If you add repositories, add the tests; this is a real gap, not a style preference.

8.5 A known bug, noted but not fixed

`src/db/repositories/compliance.ts` (around line 167) tests `ctx.role === 'admin'`, so `super_admin` **falls through to the client scope**. Super-admin compliance reads must not depend on that branch. Recorded in `docs/context/notes/design-system.md`.

---

9. Auth, routing and the API surface

9.1 How a request is authenticated

1. Auth.js issues a **JWT session** on credential login (bcrypt password check).
2. `proxy.ts` (the Next middleware) runs on `/`, `/login` and `/app/:path*`.
3. For API routes and server code, `src/auth/guards.ts` builds the `AuthContext` — and it **re-reads the role and scoping ids from the `profiles` table on every request** rather than trusting the JWT claims.

That last point is deliberate and important:

```
// Prefer live profile row over JWT custom claims (source of truth).
if (!row || row.status !== 'active') return null;
if (!isValidDbRole(row.role)) return null;
if (row.role === 'intern' && !row.internId?.trim()) return null;
```

A stale JWT — or Auth.js dropping custom credential fields, which it does — can therefore neither grant access nor block a legitimate client submit.

9.2 The middleware (`proxy.ts`)

```
unauthenticated on /app/*        -> /login?next=<path>
authenticated on / or /login     -> that role's home
role segment mismatch            -> that role's home
super_admin                      -> may enter EVERY segment (bird's-eye)
```

Two critical implementation notes:

- It imports `@/auth/edge`, **not** `@/auth/config`. The edge wrapper has no providers and no database access, which is all a request-path guard needs. Importing `@/auth/config` would pull **bcrypt and the pg pool into every guarded request**.
- Because of the segment guard, **never hard-code** `/app/manager/...` in a link. Use the helpers: `staffBase`, `adminProjectPath(eng, roleOrBase)`, `staffProjectBase`, `staffNewProjectPath`, and preferably `staffProjectBaseFromPathname` so a Super Admin working inside a manager shell stays on `/app/manager/projects`. Super Admin has **no** `/app/super/projects/*` — those helpers map super to `/app/admin`.

9.3 The guards API

```
import { requireAuth, requireRole, requireAnyRole } from '@/auth/guards';

const guard = await requireAnyRole('admin', 'manager');
if (guard.ok === false) {
  return NextResponse.json({ error: guard.error }, { status: guard.status });
}
const { ctx } = guard;
```

- Use `if (guard.ok === false)` — that exact narrowing is what lets TypeScript see `error` and `status`.
- Prefer `requireAdminOrManager()` for firm operations (create project, create leads, Knowledge Bank writes, welcome email). Reserve `requireAdmin()` for the create / list-managers APIs.
- Import from `@/auth/guards` directly. The old `@/lib/api/require-role` and `require-manager` shims were **deleted**.

9.4 The API surface (~120 routes)

Engagements / projects `GET POST /api/engagements` · `GET PATCH DELETE /api/engagements/[id]` · `/api/engagements/by-slug/[slug]` · `/api/engagements/mine` · `POST /api/engagements/[id]/restore` · `/api/admin/projects/deleted` (recycle bin) · `/api/engagements/[id]/schedule` · `/api/engagements/[id]/leads` · `/api/engagements/[id]/clients` and `/substitute` · `/api/engagements/[id]/progress-cc` · `/api/engagement-change-requests[/id]`

Checklist (the core) `POST /api/engagements/[id]/checklist` (save / patch) · `/checklist/submit` · `/checklist/review` · `/checklist/client-fill` · `/checklist/unlock` · `GET /api/checklist-index` (slim, answers stripped in SQL)

Board resolution `/api/engagements/[id]/board-resolution` plus `/generate` `/finalize` `/status` `/download` `/download-signed` `/upload-signed`

Generated documents `/api/engagements/[id]/incorporation-docs/generate` `/download` `/download-all` `/share` · `/api/engagements/[id]/doc-pack` `/[itemKey]` `/zip` · `/api/engagements/[id]/dir-2/generate` `/download`

Documents and storage `/api/documents` and `/[id]/signed-url` · `/api/engagements/[id]/milestone-documents` · `/api/milestone-documents/signed-url` · `/api/knowledge-bank` (plus `/upload`, `/folders`, `/[id]/download`, `/object`)

Compliance — `/api/filings` · `/api/filings/[id]/window`

Client portal `/api/client/overview` · `/api/client/report` (PDF) · `/api/client/change-requests` · `/api/client/steps/[stepId]/approve`

Notifications / communication `/api/notifications` (`?head=1`, `?history=1`) · `/api/announcements` (plus `/sources`, `/sources/[id]/fetch`) · `/api/email-templates` · `/api/outlook/connect` `/callback` `/send` `/directory` `/disconnect` · `/api/engagements/[id]/notification-deliveries` and `/[deliveryId]/retry` · `/api/webhooks/aws-eum` (**public, SNS-signature authenticated**)

Ask VCFO — `/api/ask/chat` `/status` `/suggestions` `/glossary` `/topics/[slug]` `/conversations/[id]` `/library` `/brief` `/nudge` `/handoff` `/admin/documents` `/admin/gaps`

Account / admin / misc `/api/account/profile` `/password` `/avatar` `/avatar/outlook` `/whatsapp-test` · `/api/admin/people` `/clients` `/interns` `/managers` `/audit-logs` · `/api/audit-logs` (scoped, used by leads and clients) · `/api/tasks` `/api/todos` `/api/requests` `/api/activity` `/api/invites` · `/api/super/overview` `/api/super/projects/[id]` · `/api/inngest` (the Inngest handler) · `/api/auth/[...nextauth]`

9.5 Route-handler conventions

- `parseJsonBody` returns the **first Zod issue message** (e.g. `intern_required`) rather than a generic `invalid_body`, so toasts can show the real reason.
- Welcome-email failures after an insert are **caught**, so a create still returns 201.
- A **404 on upload** almost always means the milestone-document routes are missing — they must stay in sync with `src/lib/milestone-document-storage.ts`.

---

10. The checklist engine — the heart of the product

This is the single most intricate part of the codebase, and the part where a careless change does the most damage. Read this entire section before editing anything named `checklist-*`.

10.1 The catalog (`src/data/checklist.ts`, 1,092 lines)

This file is **pure data plus pure helpers** — no React, no database. It defines:

```
export interface ChecklistItem {
  id: string;            // 'pre-1', 'post-9', 'reg-14'
  slug: string;          // URL segment for step routes
  bucket: Bucket;        // 'pre-inc' | 'post-inc' | 'fema' | 'statutory'
  order: number;
  title: string;
  forms: string[];       // the MCA/government forms involved
  infoRequired: string[];
  deadline: DeadlineRule;
  urgent?: boolean;
  notes?: string;              // static staff guidance, never client-submitted
  fields?: ChecklistField[];   // the form
  responsibleRole?: 'client' | 'intern';
  description?: string;
  expectedTimeline?: string;   // playbook SLA, e.g. "14–15 working days"
}
```

A field:

```
export interface ChecklistField {
  id: string;
  label: string;
  type: 'text' | 'textarea' | 'select' | 'file' | 'date' | 'segmented' | 'repeat';
  section?: string;            // groups fields into a TAB on the step page
  helperText?: string;
  required?: boolean;
  showWhen?: { field: string; value: string };     // conditional display
  labelWhen?: Array<{ field: string; value: string; label: string }>;
  filledBy?: 'client' | 'intern';                  // overrides the step owner
  layout?: 'short' | 'full';                       // desktop grid density
  // 'repeat' only:
  entryFields?: ChecklistField[];
  entryLabel?: string;   // e.g. "Director"
  emptyLabel?: string;
  minEntries?: number;   // 0 means an empty list is a valid terminal state
  maxEntries?: number;
}
```

Two field kinds are non-obvious:

- `segmented` — a `select` rendered as the app's option pills (`SegmentedPicker`) instead of a dropdown.
- `repeat` — an add/remove list of entries. See 10.4.

Phase groups are separate from items:

```
export interface ChecklistPhaseGroup {
  id: string;        // 'pre-inc-phase-1'
  title: string;     // 'Phase 1 — Name Application'
  subtitle?: string; // 'Steps 1–5'
  itemIds: string[]; // ORDER COMES FROM HERE, not from the id numbers
}
```

Key exported helpers:

Helper	What it gives you
`getIncorporationPhases()`	All four phases with their items resolved.
`getActiveCatalogItems()`	The ordered active catalog — **excludes legacy rows not in a phase**.
`getPreIncPhases()`, `getPostIncPhases()`, `getRegistrationPhases()`	Per-bucket phases.
`getPreIncPhaseStep(itemId)` etc.	`{ phaseId, stepNumber }` — the step number is the **index in `itemIds` + 1**.
`getItem(id)`	One catalog item.
`coerceStatusCode(value)`	Guards against legacy or invalid jsonb status strings.
`getChecklistStepTimelineLabel(item)`	The playbook SLA when set, otherwise the statutory deadline rule.
`copyMentionsWorkingDaysSla(text)`	Used to **filter SLA copy out of the Project Lead's UI** (see 10.9).

`StatusCode` values: `not-started`, `in-progress`, `awaiting-client`, `completed`, `overdue`, `not-applicable`.

10.2 The sequential gate — why step 7 will not open

Module: `src/lib/checklist-step-gate.ts`. Server write path: `patchChecklistItem`.

Rules:

- A step unlocks only after the **previous active catalog item** is *terminal-complete*: `completed`, `not-applicable`, a client submit, or a delivery.
- **A save-draft does not unlock anything.**
- **A Project Lead's "Request manager approval" / "Submit" does not unlock anything either.** The step reads `waiting` — "Waiting on your manager…" for the lead, "Waiting on your approval…" for staff — until the manager accepts. **A green tick means accepted, nothing less.**
- A rejected step, or one unlocked for correction, **re-locks everything after it**.
- A `not-applicable` step reads as `done` wherever it sits in the sequence (this was a bug: it used to read `locked` until the sequence reached it).
- **Overdue badges appear only on the current (active or waiting) step**, never on locked future steps.

Copy rules: the message is `"This opens after {title} is complete."` or `"Waiting on the client…"` — **never "access denied"**.

Important role exception: **the Project Lead's UI ignores the sequential lock for navigation.** A lead may open any step and the form stays editable (`allowLockedOpen`). `patchChecklistItem` does not block staff writes on the sequential lock. Only the client portal actually waits.

10.3 Step visibility — who is allowed to read what

Module: `src/lib/checklist-visibility.ts` (pure). Applied **server-side** in every route that returns `checklist_state`, via `checklistStateForViewer(ctx, state)` in the engagements repository — including `/api/checklist-index`.

> **Do not ever add a route that returns the raw blob.**

Three viewer kinds: `lead` (intern — full), `firm` (admin / manager / super_admin), `client`.

Lead to firm (`isStepReleasedToFirm`): a plain save is the lead's **private draft**. The manager sees no answers, no uploads, no tick — until one of: Request approval / Submit, Deliver to client, a client submission, an accept, or the step reaching `completed` / `not-applicable`. A pending `clientFillRequest` alone does **not** release it (the manager still sees the request itself and can decide it from the step page).

Firm to client (`isStepReleasedToClient`): a `lead_manager_request` reaches the client **only once** `reviewStatus === 'accepted'`. Under review, rejected, or after the client's own change request, the step is redacted down to `{ status, approval, clientFillRequest, … }` — the review trail, lock and `clientSubmittedAt` are stripped, and a lingering `completed` is downgraded to `in-progress` so the gate reads it as not done. `change_requested` is **not** a release; `pending_client` and `client_approved` are. **The redaction is idempotent** and test-covered.

Files follow their owning step. `listDocuments` and `getDocumentById` drop index rows on unreleased steps — the check is `checklist_state -> step_id` **in SQL**, not a full-blob load — and `/api/milestone-documents/signed-url` returns 403 by matching a path segment to a field.

On the step page itself: a manager viewing an unreleased step gets the `ClientStepFieldPreview` card ("Your project lead is still preparing this step…") plus the review panel. A client gets the same card until accepted, with only the approval status line and no Approve / Request-a-change buttons.

10.4 Repeating lists (`repeat` fields) — directors, subscribers, NIC codes

Introduced by the SPICe+ Part B restructure (2026-09-16). Module: `src/lib/checklist-repeat.ts`; editor: `RepeatGroupEditor`.

Repeat entries live **inside the step's flat `responses` object**, not in a new column:

```
responses["proposedDirectors"]                      = "e1a2b3c4,e5f6a7b8"
responses["proposedDirectors.e1a2b3c4.fullName"]    = "Jane Doe"
responses["proposedDirectors.e1a2b3c4.nationality"] = "Indian"
```

Rules that will bite you:

- Entry ids are `e` + 8 hex characters.
- **Removing an entry must write `''` to each of its keys.** The patch merge keeps unknown keys, so a delete has to be an explicit blank.
- **Never key anything by entry index.** Ids survive reorders and removals; indexes do not.
- `showWhen` / `labelWhen` inside an entry template name the **sibling relative id**; `expandRepeatEntry` re-points them to the concrete dotted id. Call `applyShowWhen` then `resolveFieldLabels` on the expanded fields **everywhere they render** — editor, read-only record, repeat cards, attachments.
- Validation errors: min/max on the **group id**, per-field errors on the **dotted id**. A section counts a group as pending when any error key equals the group id or starts with `${groupId}.`.
- `pre-16` (Subscriber Details) **submits validly with zero subscribers** — "No subscribers to add" is a real terminal path (`minEntries: 0` + `emptyLabel`), not a missing-field state.

10.5 Per-step validation

Each step has its own validator module, colocated with its test:

```
src/lib/checklist-pre1-validation.ts      (+ .test.ts)
src/lib/checklist-pre6-validation.ts
src/lib/checklist-pre7-validation.ts
src/lib/checklist-pre8-validation.ts
src/lib/checklist-pre9-validation.ts
src/lib/checklist-pre10-validation.ts
src/lib/checklist-pre11-validation.ts
src/lib/checklist-pre12-validation.ts
src/lib/checklist-part-b-validation.ts
src/lib/checklist-step-validation.ts      <- the SHARED validator used by both
                                             the form and the submit route
```

`src/lib/checklist-step-validation.ts` is important: **the form and the server run the same function**, so a client can never submit something the form would have rejected, and the server's 422 lands in the same error display.

`validatePre1Responses(responses, { visibleFieldIds })` requires **only what is actually rendered** — because `pre-1`'s field set varies with ownership type.

10.6 Pre-incorporation direct flow (2026-10-13) — the newest major change

Spec: `docs/specs/pre-inc-direct-flow/` (CONTEXT, AUDIT, REPORT). **One policy module: `src/lib/pre-inc-direct-flow.ts`.**

The firm decided the manager-approval step was unnecessary friction for pre-incorporation data entry. So:

- `DIRECT_FLOW_STEP_IDS` = **every active pre-incorporation step except `pre-2`**. Never write that step-id list anywhere else.
- **Who edits:** the client *and* the Project Lead fill the **same tabbed form** and either one can submit. Manager / admin read it and may "Reopen step".

- **Lead-only fields:** `pre-1`'s `nicCode`, `nicBusinessType` and `additionalNicCodes` (including its dotted entry ids). On a direct-flow step, `filledBy: 'intern'` no longer *hides* a field from the client — it shows a read-only row saying "Your project lead fills this." **The server strips lead-only keys from client patches and submits.**
- **Submit** (`POST .../checklist/submit`, `submitDirectFlowStep`): merge → shared validator → 422 with `fieldErrors` for anything the caller can fix → if a client has only lead-only gaps left, the step becomes `waitingOnLeadPatch` (gate reads `waiting` for the client, `active` for the lead) → otherwise `directSubmitPatch` stamps `completed`, `reviewStatus: 'accepted'`, `clientSubmittedAt`, `deliveredToClientAt`, `submittedByName` / `submittedByRole`. **`pre-12` copies the incorporation date to `engagements.incorporation_date` here.**
- **Reopen:** the unlock route and the client's "Request a change" write `directReopenPatch`, which clears the submit stamps. There is **no per-field unlock list** — those cannot name repeat entry ids.
- **Read-time rules for old data** live in `isDirectFlowStepComplete`: accepted / completed / pending lead request / pending client submit all read complete; rejected or reopened read open; `approval` and `clientFillRequest` are ignored. `isChecklistStepSequentiallyComplete` takes an **optional `itemId`** — **pass it**, or direct-flow steps are read with the old rules.
- **Visibility:** direct-flow steps are released to the firm and the client at all times. `notes` is still stripped for the client. `pre-2` keeps the old rules.
- **Gate:** a client may open a later direct-flow step **read-only** ("This opens after … is complete."); the server keeps `sequentialLockMessage`. Client writes to a submitted or waiting step get **409**.
- **No approvals exist on direct-flow steps.** `RequestManagerApproval`, `ChecklistReviewActions`, `RequestClientFill` and `ClientStepApproveButton` render nothing there, and `reviewChecklistItem`, `requestClientFill`, `decideClientFillRequest` and `clientApproveStep` all throw `step_has_no_approval`. Every "pending manager decision" reader goes through `isAwaitingManagerDecision(itemId, slice)`.
- **Board resolution (`pre-2`) is the one surviving approval:** the lead clicks "Send for manager approval" (the submit route plus `requestBoardResolutionApproval`, which requires the draft file to exist and the generation checks to pass) → the manager clicks "Approve and send to client" (`approveBoardResolution` = finalize + close `pre-2`) or "Send back" (a note is required, and only the lead hears it). While pending, `saveBoardResolutionDraft` and the generate route refuse the lead. Manager UI: `ManagerBoardResolutionReview`.

New `checklist-state-key.ts` slice keys from this work: `submittedByName`, `submittedByRole`, `waitingOnLeadAt`, `waitingOnLeadByName`, `lastEditedAt`, `lastEditedByName`, `lastEditedByRole`.

10.7 Part A, NIC codes and manager date windows

- **Part A sections live in exactly one helper**: `src/lib/part-a-sections.ts` (`PART_A_SECTION_ORDER` in MCA order, `partAsectionsFor(ownershipType)`, `partAFieldsFor(...)`, `isParentEntityField`). `fieldsForOwnership` in `checklist-responses.ts` delegates to it for `pre-1` only. The form hook, client preview, rail attachments and completeness all read the returned field list. **Never hard-code a tab list or a tab count.**
- **NIC code**: `nicCode` is 5 digits and required. `nicBusinessType` is auto-filled in `setField` from `src/lib/nic-2008.ts` (1,302 sub-classes parsed from the official MSME PDF into `src/data/nic-2008.json` as `[code, subclass, class]`). An unknown code is a **warning, not an error**. `nicBusinessType` renders read-only and is stored in responses. `pre-1` also has an "Add NIC code" repeat (`additionalNicCodes`, max 9); the main one is labelled "Main NIC code"; all of them print in the board resolution.
- **Manager date windows**: `src/lib/schedule-windows.ts` (`windowForStep`, `applyScheduleWindow`, `formatWindow`, `canSetScheduleWindows`), repository `src/db/repositories/schedule.ts` (manager / admin / super only — it throws otherwise), routes `POST /api/engagements/:id/schedule` and `POST /api/filings/:id/window`. Stored in `engagements.schedule` (jsonb, its own column) and `compliance_instances.window_*`. UI: `ScheduleWindowControl` for writers, `ScheduleWindowMeta` for readers. The old "Typically takes N working days" SLA line in the client's next-action is **gone** — the window shows instead. After a save, invalidate `['engagements']` and `['filings']`.

- Today's filings join DB windows by the composite key `${engagementId}:${obligationId}:${dueDate}:${periodLabel}`.

10.8 Lead autosave — what keeps a half-filled step alive

This is in `useMilestoneResponseFormState` and it has been the source of two genuinely nasty data-loss bugs. Read before touching.

- `allFields` is **memoised on** `item`. It used to be rebuilt every render, which re-created `flushPendingAutoSave` → `clearDebounce` → the unmount-flush effect's dependencies, so **that effect's cleanup ran on every render**: it cleared the 600 ms debounce before it could fire and flushed against refs that had not synced yet. One edit plus a refresh lost the edit; slow typing posted duplicate partial words. **Do not key that effect on callbacks again.**
- The flush effect is keyed on `autoSaveEnabled` **only**, and reads the latest flush through a ref. It fires on unmount, on `visibilitychange → hidden` (tab switch) and on `pagehide` (refresh / close, using `fetch keepalive`). It flushes only when `userEditedRef` is set — otherwise a remount posted the `pre-1` engagement defaults back as if they were answers.
- **Draft =** `saved` **overlaid with** `touched` (`DraftEdits { values, touched }`, `overlayTouchedFields`). The step page renders **before** the full checklist loads (the slim index has no answers), so a full snapshot taken at that moment showed empty fields and the next autosave diff sent `''` for every server-held field — a complete wipe. Untouched fields always follow `saved`; the diff only ever carries touched fields.
- `engagementsSettled` (in AppContext) is true **only once the fetched list is applied to state** (`appliedEngagementsData === engagementsQuery.data`), not on the query's success render. Step and project pages redirect on "settled + no engagement", and that one-render gap used to bounce roughly **1 in 3 hard refreshes** of a step page to the clients list.
- Autosave debounce is ~600 ms, through the same `updateItem` path as Save. Save is hidden while clean and shown while pending / saving / failed.
- **Autosave must never re-send the manager-approval email.** Intern draft persist uses the same `POST /checklist` as Save but sends only `{ responses }`. Notify only when the PATCH itself carries `reviewSource=lead_manager_request` (Request / Submit) or `resendManagerEmail` ("Email manager again"). Helper: `src/lib/email/lead-manager-request-notify.ts`. Otherwise every field debounce stacked an identical "manager approval requested" toast and email.

10.9 Server-side write guards

- `POST /checklist` runs a Project Lead's patch through `leadWritablePatch`, which drops `deliveredToClientAt`, `completedOn`, `reviewedAt` / `reviewedBy`, `approval`, any `reviewStatus` other than `reviewing`, and `status` values `completed` / `not-applicable`. **No client build can release a step on the lead's say-so.**
- There is **no lead "Deliver to client" action any more** (removed 2026-09-15). The manager's Accept *is* the delivery: `reviewChecklistItem` stamps `deliveredToClientAt` and, for `pre-12`, copies `responses.dateOfIncorporation` onto `engagements.incorporation_date`. `validateInternDelivery` (the `pre-4` / `pre-5` required uploads) now runs inside the lead's **Submit**.

10.10 Missing-field display (2026-10-13)

- `SHOW_INTERN_FORM_ERROR_SUMMARY` is now **true**.
- Errors appear for a tab only after the user leaves it with **Next**, and for every tab after a **failed Submit** (then live on every change).
- Tabs with errors carry a red dot plus an `sr-only` "Needs attention"; fields get `border-danger`, `aria-invalid`, `aria-describedby`; each summary line opens the tab and focuses the field.

10.11 Form layout rules

- Checklist fields **pair on desktop**: `.milestone-form-grid` is 2 columns from `md`. Short = text / date / select. Full-width (`grid-column: 1 / -1`) = textarea, file, address text, or helper copy longer than ~120 characters. An odd leftover short field stays in one cell (~50%), not stretched. Phones get one column. Inferred by `getMilestoneFormFieldLayout`; the optional `ChecklistField.layout` overrides it.
- File upload is a **compact ~44 px row** (`.milestone-upload-zone`, max 88 px), not a tall centred dropzone.
- Remarks use `.milestone-form-textarea` (min-height 72 px / 3 rows, grows with `field-sizing: content`).
- Section tabs stay on **one row** (`overflow-x`), map vertical wheel to `scrollLeft`, and use edge arrows in dedicated 36 px slots.
- File fields POST to `/api/engagements/:id/milestone-documents` and then PATCH the storage path immediately.

10.12 The one date input in the app

`NoirDatePicker` is **the only date input**, used by every checklist `type: 'date'` field. It is type-or-pick: a typed `DD/MM/YYYY` with automatic slashes (ISO and `15 Sep 2026` also parse), a month grid plus a scrollable year grid (1940 to +30 years) behind the caption, and Clear / Today in the footer.

Known library trap: **react-day-picker v10 marks the `<td>` as selected, not the day button.** `aria-selected:` utilities on `day_button` never match, and the `ghost` button variant's `text-primary` paints the selected day blue on blue. Use a parent selector on the button (`[[aria-selected=true]_&]:text-primary-foreground`), as `NoirDatePicker` does.

---

11. Generated documents — the biggest time-saver

Three related systems, often confused. Know which one you are in.

System	What it is	Where
Board resolution (`pre-2`)	The parent company's resolution authorising the Indian entity. Editable draft, manager approval, finalize, signed upload.	`src/lib/board-resolution*.ts`, `src/storage/board-resolution.ts`, `src/components/board-resolution/`
Incorporation drafts (`pre-7` / `pre-8`)	Ten-plus per-director and per-company legal documents generated from already-collected answers, then shared to the client for signing.	`src/lib/incorporation-docs/`
Document pack	A **read-only model** over everything above: what exists, what is missing, what unblocks fastest. Staff-only.	`src/lib/doc-pack/`, `src/components/doc-pack/`

11.1 How generation works

Templates are real `.docx` files with **docxtemplater tags**. The app fills them from `checklist_state` answers and streams the result. Rules:

- Templates are **re-tagged by scripts, never edited by hand**: `scripts/incorp-docx-retag.mjs`, `scripts/incorp-docx-build-declarations.mjs`, `scripts/incorp-docx-build-sourced-fields.mjs`, `scripts/prepare-noc-board-resolution-docx.mjs` (all idempotent; they share `scripts/docx-comment-tags.mjs`). **Edit the script, not the .docx.**
- `renderDocx` takes **array data only for declared loop keys**. The DIR-8 prior directorships table is a row loop (`PRIOR_DIRECTORSHIPS`).
- Generators consume the **legacy `pre1` / `pre6` shapes**. Do not pass raw step responses to them — go through **`directorResponsesFromState(state)`**, which overlays `pre-15` entries as `director{n}*` / `residentDirector*` / `nrDirector*` slots and `pre-14` registered office. `directorsAccepted` in `incorporation-docs-errors.ts` is the acceptance gate.

11.2 Per-director audience keys (the slot system)

`src/lib/incorporation-docs/audiences.ts` is the **only** place that builds these:

Director	Audience key	Field prefix
First non-resident	`non-resident`	`nrDirector`
First resident	`resident`	`residentDirector`
nth non-resident	`non-resident-{n}`	`nrDirector{n}`
nth resident	`resident-{n}`	`residentDirector{n}`

Slot numbering is **per residency, in list order**, matching `directorsAsPre6Responses`. Draft field ids are `{prefix}{docSuffix}DraftUrl`, signed ids `{prefix}{docSuffix}SignedUrl`. Company documents keep fixed ids. Slot-1 ids, row keys and S3 paths are unchanged from before the feature.

Other rules:

- `pre-7` / `pre-8` fields are a **static superset**: `expandDirectorSlotFields` declares slots 2–6 after each slot-1 field; the form shows only the directors actually on file (`fieldsForDirectorAudiences`), and read-only lists hide later slots until they hold a value (`withoutUnusedDirectorSlots`). `extractItemResponses` keeps **only declared ids** — a new per-director id must be declared or it is silently dropped on read.
- **Frozen slot set**: once `pre-7` is shared (`incorpDraftsSharedAt`) or accepted, slots that were not in the legacy set become optional (`isIncorpSlotSetFrozen`). `pre-8` does the same once accepted.
- ⚠️ **Renumbering hazard, not fixed**: removing or reordering a director renumbers the slots, so drafts and uploads stay under the **old** key — `resident-2` might now be a different person. Stable entry ids would fix it; it has not been done. **Regenerate after changing the director list.**
- Generators render every director who has a residency, up to six per kind. `evaluateDocPack` lists directors **without** a residency, or past the ceiling, in `summary.skippedDirectors`.

11.3 What the generators produce

Company-level: board resolution (foreign / Indian-investing / Indian-name-use variants), authorisation letter, acceptance letter, MOA (INC-33), AOA (INC-34), AGILE-PRO-S (INC-35), subscription sheets (body-corporate vs individual layout, chosen by `planSubscription` from `pre-16`), NOC letters.

Per-director: DIR-2 consent (prints DIN, other-interest count, CS membership), DIR-8 declaration (with the prior-directorship row loop), INC-9, deposit declaration (optional), ID & address declaration (DIN holders only, optional).

Niceties that were deliberately built and should not regress: "daughter of" / D/o for female directors; AOA per-director salutation that no longer lists the NR / resident director twice; MOA clause II state and clause V shares read from `pre-13`; share counts described in words; parent country/state resolution via `src/lib/parent-jurisdiction.ts` (old address parsing is only a fallback).

`DocDefinition.appliesToEngagement` drops the board resolution, authorisation and acceptance letters for **Independent** companies.

11.4 The document pack

- Staff-only. Routes: `GET /api/engagements/[id]/doc-pack`, `/[itemKey]` (`?preview=1`), `/zip` (built with pizzip).
- Pages: `/app/intern/engagements/[id]/documents` and `{staffBase}/projects/[slug]/documents`.
- Pack item keys are `{docId}:{audience}` — the same `doc:audience` row keys `pre-7` uses for `sharedIncorpDraftDocs`, so `incorpDocRowKey` maps both ways. **URL-encode the colon** in links (`docPackItemUrl`).
- The pack **never writes** checklist state or S3, and **never serves a board resolution draft**. An attached `pre-7` file wins over an on-demand render.

- It links to a form tab by `sectionSlug(field.section)` and the step page reads `?tab=<slug>`. An unknown slug leaves the first tab selected. **Never store a tab index** — Part A's tab set varies with ownership type.
- `useDocPack` is invalidated on checklist save and on board-resolution finalize.
- Status: **stopped at the Phase 5 checkpoint** before 5c. The `pre-7` panel was reduced to one "Generate & attach for filing" action; the document pack page is now the only place drafts are listed or generated.
- `src/lib/api/doc-pack.ts` imports the auth guard, so **any test importing it must mock `@/auth/guards` and `@/db/repositories/doc-pack`** (next-auth pulls in `next/server`).
- Formatting oddity to preserve: `src/views/intern/useBoardResolutionEditorState.tsx` has a blank line after every line. Keep that format or the diff doubles.

11.5 Board resolution specifics

- The template is chosen by **NOC variant** (`boardResolutionVariant` in `src/lib/board-resolution.ts`): foreign → `boardResolution.docx`; Indian investing or Indian name use → `boardResolution-indian-*.docx`. Standalone, or an unanswered Indian role, falls back to the foreign template.
- Generation **stops on a missing NIC code** and shows a linked missing-field list (`missingFieldRefs`).
- `POST .../board-resolution/finalize` is the **only release action**: it sets `status=finalized`, marks `pre-2` `deliveredToClientAt` (which unlocks `pre-3`, the signed upload), and fires `notifyEngagementEvent({ event: 'board_resolution_shared' })`. The route is manager / admin / super-admin only and returns **409** when not pending.
- Re-clicking "Send to client" reopens the compose window without duplicating in-app notification rows. The client GET already hid drafts until finalized.
- Client-side, `pre-2` / `pre-3` show the document on a **stage with the approval actions under it** (`src/components/client/ClientStepDocument.tsx`).

---

12. Compliance calendar and filings

12.1 The two kinds of "compliance" in this app

Do not confuse them:

1. **The statutory master calendar** — `src/data/statutory-calendar-fy2627.ts` (`STATUTORY_DEADLINES`). A **fixed FY calendar of Indian statutory deadlines**. It has **no status, no owner and no period column**. The list view *derives* those three for display only:
   - `statutoryStatus`: past = overdue, within 7 days = due soon, else upcoming. **There is no `filed` state here.**
   - `statutoryReturnPeriod` / `statutoryFilingName`: parsed from the trailing parenthetical of the title.
   - "Mine" = applies to any engagement in the passed portfolio, via `deadlineAppliesTo`.

Do not add status / owner / period columns to that data file.

1. **Generated filing instances** — `compliance_instances` rows, produced by the Inngest generator from `compliance_obligations` plus `engagement_compliance_triggers`. These **do** have status, owner, due date and a period label, and they are what `/api/filings` returns.

12.2 Is the company incorporated?

```
isIncorporated(engagement, checklistState?)   // src/lib/compliance/incorporation-state.ts
```

This is **the only source of truth** for "COI recorded". It checks `engagements.incorporation_date` first, then falls back to `pre-12` being terminal (via `isChecklistStepSequentiallyComplete`). Since 2026-09-15 it is also true whenever `stage`

`!== 'Pre-Incorporation'`. **Do not hand-roll this check** — the super-admin summary's `incorporated` flag goes through it too.

Compliance pages are **visible before incorporation**, showing `PreIncorporationNotice` (which owns all the client/staff copy and composes `ui/alert` + `IconChip` + the login info tint). The caller decides *whether* to show it, from `isIncorporated`. The shared Calendar / Filings views take a `preIncorporation` prop and keep their normal layout in the genuine empty state — **nothing is seeded to fill the space.**

12.3 The shared compliance views

`src/views/compliances/*` are the **only** compliance surface for **every** role, rendered by one wrapper (`CompliancePages.tsx`) with one `ComplianceScope`. The chrome is identical in every shell. The company picker and the Company column appear **only when the roster has more than one company** (`soleCompany`). `scope.audience` picks notice wording and **nothing else**.

> **Do not add role branches to these views**, and do not resurrect `views/admin/Compliance.tsx` or `views/client/Compliances.tsx` — both were deliberately retired.

Where each shell's Compliances pages live is decided once in `src/components/shell/compliance-nav.ts` (`compliancesBasePath`, `complianceLeaves`). **Super admin points at `/app/admin`** — there is no `/app/super/compliances`.

Staff filings state lives **in the URL**: `?company=<engagement id>` and `?status=upcoming|due-soon|overdue|filed`, alongside `cadence` / `period` / `fy` / `view`. The calendar carries `company` across its links.

`buildMatrix(..., { perCompany: true })` keys rows by engagement — **without it, two companies' identical obligations merge into one row.**

12.4 The statutory calendar UI

- Every role's `/compliance` page is the **statutory calendar only** — no PageHeader, no KPI row, no pill tabs. The filing tracker lives separately at `<base>/compliance/tracker`.
- Layout: the month's deadlines grouped by date on the left (`.stat-cal-list`), a compact month navigator plus legend on the right (`.stat-cal-nav`, sticky from `lg`). **The legend rows are the category filter** (they drive `mutedActs` via `toggleMutedAct`). The old FY heat strip and act-chip toolbar are gone.
- **Two colour lanes, never blended.** *Category* (which law) = `--stat-*` from `ACT_SWATCH`, used only on a row's code tag and the calendar dots. *Status* (state of the filing) = danger / accent-teal / primary-light, used only on the pill at the right of a row. Act hues are kept at least 45° apart; do not reuse IconChip emerald/teal or rose/pink, which merge as adjacent dots.
- Calendar cells carry up to **3** category dots in catalog order (`statutoryCellActs`), over three fixed slots so every cell keeps the same height. Days with more acts say the true count in their `aria-label` only; the left-hand list is the full record. Clicking a dated cell smooth-scrolls to `#statutory-agenda-YYYY-MM-DD` and flashes that group.
- Deadline scoping: admin and super see the **full master calendar**; managers and leads see only deadlines that apply to a client in their role-scoped portfolio.
- **"Remind me" is deliberately a disabled button** — there is no reminder backend for statutory deadlines. Wire it only alongside real delivery.

12.5 Known caveats

- The **staff filing tracker** uses `computeAllFilings` (in-memory `generate-instances`), whose `fixed_annual` obligations anchor on `'2000-01-01'` when there is no incorporation date. So a pre-COI company can show a couple of "generated" rows there while the DB-backed `/api/filings` register (what the client sees) has none. **This is a known, accepted difference** — an owner call, not a bug to fix silently.

- Staff month sheets can be **honestly empty**: the demo FY 2026-27 register has no monthly-frequency instances at all (quarterly / annual / half-yearly only).
- The **Project Lead compliance gate**: `complianceEngagementsForRole('intern', …)` filters to incorporated engagements. That drives the sidebar (the Compliances group hides when empty), `useInternPortfolio`, and a redirect to `/app/intern/today` when a lead has none. Other roles are untouched.
- The compliance generator is a **system job**: call `systemGenerateComplianceInstances` / `runComplianceGenerate` **from Inngest only** — those functions use `db` without an `AuthContext`. Obligation FK rows are upserted from `COMPLIANCE_OBLIGATIONS` on the first run.
- **The full email digest from the compliance job is deferred** — today it only logs to the console.

---

13. Documents and object storage

13.1 Two parallel document systems

System	Where bytes live	Where the reference lives	Download route
Milestone documents (checklist file fields)	S3 prefix `milestone-documents/`	the step's `responses` in `checklist_state`	`/api/milestone-documents/signed-url?path=…`
Indexed documents (the vault)	S3	a `documents` table row	`/api/documents/[id]/signed-url`

Staff and lead milestone uploads **also** call `createDocument`, so the vault index is complete. **Client uploads stay storage-only** in `checklist_state`.

Other prefixes: `avatars/{userId}/photo` (profile photos, migration 0011), plus the board-resolution and incorporation-doc keys.

13.2 Upload flow

The browser POSTs **multipart** to `/api/engagements/:id/milestone-documents` with `fieldId` + `file`. The server authenticates, runs `assertEngagementAccess`, then writes to S3/MinIO. The client then PATCHes the storage path onto the step immediately.

Routes must stay in sync with `src/lib/milestone-document-storage.ts` — **a 404 on upload has almost always meant one of those routes was missing.**

The upload size limit constant is `MAX_UPLOAD_BYTES` from `@/lib/upload-limits`, surfaced to the browser as `NEXT_PUBLIC_MAX_UPLOAD_MB` (default 50).

13.3 Download authorisation

`/api/milestone-documents/signed-url` **403s by matching a path segment to a field** — that is how it refuses a file belonging to a step the caller may not see. `listDocuments` / `getDocumentById` drop index rows for unreleased steps using a `checklist_state -> step_id` SQL expression (not a full blob load).

13.4 The vault and the Knowledge Bank

- **Vault** (`/app/{intern|manager|admin}/vault`, shared view `src/views/vault/DocumentVaultPage.tsx`): files grouped by assigned company, then phase (pre-inc / post-inc / FEMA / statutory), then checklist step. Search matches the file name and the company. Lead isolation is Path A in `listDocuments` / `getDocumentById` — **a lead cannot see another lead's client files.**
- **Knowledge Bank** (`/api/knowledge-bank`): the firm's own reference library, with nested folders. `GET` returns `{ files, folders, tree }`; an optional `?folderId=` (or `root`) adds `{ current: { folder, ancestors, folders,`

`files } }`. The list DTO includes `folderPath` for the command palette and global filename search. **Clients have no access at all.**

- **CommandPalette** (debounced) uses the same Path A `GET /api/documents` and `GET /api/knowledge-bank` lists, so a lead searches only assigned companies, a manager their clients, an admin everything. Clicking a result goes to the vault or KB with `?q=`.
- Demo-data detail: the APIs map UUIDs to `e1`, and matching uses `engagementIdAliases` so indexed files are not dropped.

---

14. Email — three paths, one dispatcher

Understanding which of the three email paths you are in prevents most email bugs.

Path	Who it is from	Transport	Used for
1. System / process mail	the app, as the company	**Resend** (default) or **AWS SES**	client-to-lead submissions, uploads, lead-to-manager approval requests, welcome mail, password info
2. Staff-composed mail	the actual person's Outlook mailbox	**Microsoft Graph** `Mail.Send`	lead-to-client and manager-to-client messages
3. In-app compose draft	the app opens a compose window, the person sends it	Graph	manager Accept, review reject, deliver, share, unlock, board-resolution finalize

14.1 The dispatcher

```
src/lib/email/send-email.ts      <- sendEmail(), the ONLY entry point
  ├── send-via-resend.ts         (EMAIL_PROVIDER=resend, default)
  └── send-via-ses.ts            (EMAIL_PROVIDER=ses)
```

Do not add a second Resend or SES call site. Import `sendEmail` and `SendEmailResult` from `@/lib/email/send-email`. `sendResendEmail`, `SendResendResult` and `resolveResendDevRedirect` **no longer exist** — they were deleted in the dead-code sweep.

With no configured From address, no Resend key, or no SES identity, sends **console-skip** rather than throwing. That is how local development works.

14.2 From and Reply-To rules (deliberate, do not simplify)

- **Client → lead process mail** is sent From `{sanitized-company-name}@sbctrack.in` — e.g. `Acme Pvt Ltd <acme-pvt-ltd@sbctrack.in>` — **so Project Leads can filter by company in Outlook.** Reply-To is the client.
- **Lead → client is not Resend at all**: the app opens a compose window and Graph sends it from the lead's own linked Outlook mailbox.
- Display names can be overridden per send to show who initiated the mail, e.g. `"Priya Sharma via VCFO Suite <info@sbctrack.in>"`.
- A Resend `FROM` on `@resend.dev` **can only deliver to the Resend account owner**. For real delivery, verify a domain.

14.3 The process-event fan-out

```
notifyEngagementEvent / notifyEngagementEventBackground
  -> src/lib/email/notify-engagement-event.ts
  -> recipients from src/db/repositories/engagement-recipients.ts
```

Wired on: checklist submit, review, unlock, deliver; incorporation-doc share; document-request create; board-resolution finalize.

Who gets what:

- **Client submit / client document upload** → the **lead plus every project manager**. Managers are resolved from `engagements.manager_id`, then `engagement_managers` membership, then the lead's `reports_to_manager_id` if both are empty.
- **Lead "Request manager approval" / "Email manager again" / "Submit"** → set `reviewSource=lead_manager_request` and email **managers only**, from the lead's Outlook when connected, else Resend.
- **Manager Accept** → opens a Graph compose **to the client**, CC firm admins plus leads, plus the engagement progress CC.
- Re-requesting after an accept reopens review and emails again.
- **Lead resolution requires `profiles.intern_id` to match `engagements.intern_id`.**

In-app bell rows for these events are inserted server-side with `createNotificationsForUsers`. Client checklist diffs toast and invalidate the bell but **do not re-persist those kinds** (that would duplicate rows).

14.4 Direct-flow mail (2026-10-13)

Event	Mail
`step_submitted` by a **client**	Resend to leads + managers, From `{company}@sbctrack.in`, Reply-To the client
`step_submitted` by the **lead**	Resend to the clients, managers on CC (`directFlowEmailPlan`)
`step_waiting_on_lead`	Resend to **leads only**
`step_reopened`	**no mail at all**
Board resolution "Send back" (`review_rejected` on `pre-2`)	no client compose, no client bell — Resend to the leads with the manager's note
Board resolution "Approve and send to client"	the existing `board_resolution_shared` compose to the client

14.5 Outlook / Microsoft Graph

- Azure app needs delegated `Mail.Send`, `User.Read`, `offline_access`. Redirect URI: `{AUTH_URL}/api/outlook/callback`.
- Env: `AZURE_AD_CLIENT_ID`, `AZURE_AD_CLIENT_SECRET`, `AZURE_AD_TENANT_ID` (the firm tenant, or `common`).
- Connect at `/api/outlook/connect` or from Settings. Tokens are stored in `outlook_connections`, **encrypted with `AUTH_SECRET`**.
- **It is not an Auth.js login provider** — it is a separate mailbox link.
- On AWS the redirect `https://app.sbctrack.in/api/outlook/callback` **must stay registered** on the Azure app.
- The profile-photo copy (`POST /api/account/avatar/outlook`) reuses the same Graph token (`User.Read` is already in the scopes). Upload overwrites the Outlook copy and vice versa — last write wins.

14.6 The staff compose page

`/app/{intern|manager|admin}/mail`.

- **To** is one compact row: chips plus name search, with **Team** (reports-to manager) and **Client** (engagement company) selects beside the field. Picking a Client writes that company's client `profiles.email` in as a visible **address chip**, not a name-only pill. Changing or clearing Client swaps only those auto-added chips.
- The directory resolves clients via `engagements.client_id` → `profiles.client_id` when `client_user_id` is empty.

- To also accepts **typed addresses outside the directory** (`src/lib/email/recipient-input.ts`): Enter / comma / Tab / blur / paste turn text into chips; a typed directory address selects that person instead; Backspace on empty text drops the last chip. `?to=` with an unknown address becomes a chip too.
- The directory returns inactive people too; To defaults to **Active**.
- **Templates** reuse `email_templates` with a `branding` column (`sbc` | `plain`). CRUD at `/api/email-templates`. Leads edit and delete **their own** rows; admin and manager may touch any.
- Send: `POST /api/outlook/send` with `templateId` / `branding`. **If `templateId` is set, branding is loaded from the DB — never trusted from the client.** `sbc` wraps the HTML with `renderEmailDocument({ brand: 'sbc' })` (`src/lib/email/compose-branding.ts`), hosting `public/sbc-logo-light.png` via `siteUrl()` (no CID, no Resend attachment). An untemplated send stays `plain`.
- Email toasts use a stable `email-dispatch:…` id so a retry **replaces** rather than stacks. The undo toast id is `notification-undo` (top-right, 7 s).

---

15. WhatsApp notifications

15.1 The principle (do not violate it)

> **WhatsApp is an outbound-only *nudge* channel. Email is the system of record. A WhatsApp skip or failure must never touch, slow, or downgrade the email path.**

Also: **never put a CIN, PAN, TAN, filing number, password, OTP or document content in a template variable.** A WhatsApp message says something needs attention and links to the portal. Passwords stay in email only.

15.2 One transport, on purpose

```
sendWhatsAppTemplate      (dispatcher — runs ALL the guards)
  -> sendViaEum            (src/lib/notify/send-whatsapp-eum.ts)
     shared types in send-whatsapp-shared.ts
```

AWS End User Messaging (Social) is the only transport. It was chosen so the Meta per-message fee lands on the **same AWS invoice** as SES, RDS and S3. A Twilio sandbox transport was added on 2026-10-12 and **deliberately removed on 2026-10-14 by the owner**. `WhatsAppConfig` is `{ enabled, eum }`.

> **Do not reintroduce a provider field "just in case."**

EUM needs **no credentials in config** — it authenticates through the App Runner / ECS **instance role**, like SES. So `isWhatsAppConfigured` checks only `WHATSAPP_ENABLED` and `EUM_PHONE_NUMBER_ID`.

15.3 Guard order (load-bearing)

```
disabled -> no_template -> no_phone -> no_consent -> opted_out
```

`resolveWhatsAppChannel` is **pure** and returns the approved Meta template name as `templateRef`, which lands in the text `template_sid` column (the column name was kept; it holds a name, not a Twilio SID). **The guards run once, in the dispatcher** — not scattered.

A recipient needs `profiles.phone_e164` **and** `whatsapp_opt_in_at` (Settings → Edit → tick consent).

15.4 Recipients mirror the email exactly

Each WhatsApp goes to the people on **that same mail** (To, plus CC where the mail copies someone), **minus the person who triggered it** (`mailHandlersForEvent`). Someone with no phone number or no consent is skipped.

15.5 The fourteen templates (LCC parity, 2026-10-14)

Bodies and variable order: `docs/context/notes/whatsapp-templates.md`. Mapping: `whatsappEventFor` in `src/lib/notify/process-event-whatsapp.ts`.

To the client: `step_awaiting_approval`, `client_action_needed` (from `client_fill_approved`), `documents_shared`, `document_delivered`, `coi_issued`, `welcome`, plus three compliance events (`compliance_due_monthly`, `compliance_due_quarterly`, `compliance_overdue`).

To the firm: `staff_client_submitted`, `staff_client_uploaded`, `staff_change_requested`, `phase_approved` (whose email goes to leads and managers only).

Everything else falls back to `process_notice`.

`inAppOnly` and `skipInAppNotifications` (a re-opened compose) send nothing.

Template names default to the event name, so no per-event env var is needed — approve them in WhatsApp Manager under exactly `welcome`, `coi_issued`, … category **utility**, language `en`. Override only with `WHATSAPP_TEMPLATE_NAME_<EVENT>`.

15.6 Reliability mechanics

- **At most once a day per person per event.** Each queued send carries the Inngest event id `wa:<event>:<engagement>:<step>:<person>:<YYYY-MM-DD>` — Inngest drops a repeat, so a double click or a re-run cannot message twice that day.
- **Throttle:** the `whatsapp-send` function is `throttle: { limit: 20, period: '1m' }`. Excess runs wait.
- **Retries happen *inside* the `send` step.** A step that *returns* is memoised, so throwing after it only replayed the stored failure. The step now **throws** on a retryable code, and the last attempt records the failure.
- **Status only moves forward** (`src/lib/notify/delivery-status.ts`): `queued 0 → sent 1 → delivered 2 → read 3`; `failed 4` is final; `skipped` is final. The webhook update matches only rows one step behind, so a late or duplicate SNS event changes nothing.
- `isRetryableWhatsAppError(code)` classifies on the SDK exception name (`ValidationException`, …) and on Meta's numeric codes. **Unknown codes stay retryable** — a misclassified outage costs three attempts, while a misclassified permanent error would silently drop a real notice.
- **Meta wants bare digits**: `toMetaPhone` strips the `+` for the EUM payload; `fromMetaPhone` adds it back when a webhook reports a number, because profiles are keyed by E.164.

15.7 The SNS webhook

`/api/webhooks/aws-eum` is **public — the SNS signature is the authentication.** `aws-sns-verify.ts` rebuilds the canonical string per message type, **refuses any signing certificate that is not HTTPS on `sns.<region>.amazonaws.com`**, and verifies RSA-SHA1/SHA256 per `SignatureVersion`. Set `AWS_EUM_SNS_TOPIC_ARN` to **pin the topic**: a valid signature proves the message came from SNS, not that it came from *your* topic.

Parsing trap: the SNS `Message` holds `whatsAppWebhookEntry` as a **JSON string inside the JSON body — you must parse twice.** `eum-events.ts` is pure and tolerates both nesting forms. An unparseable event is acknowledged with **200, never a 500** (SNS would just retry a payload that can never parse).

Still outbound-only: the parser returns the sender and an opt-out verdict and nothing else. No inbound message body crosses that boundary, is logged, or is stored.

15.8 The Send log and retry

- Project page, **admin + manager**: `SendLogPanel` reads `GET /api/engagements/:id/notification-deliveries`, with channel and status filters. Phone numbers are masked (`+91 98480 •••00`, `maskPhone`); the reason column uses `skipReasonText` / `whatsappErrorText`.
- **Retry**: `POST …/notification-deliveries/:deliveryId/retry` (admin + manager, same scope as the GET) re-queues a failed WhatsApp row with dedupe id `retry:<deliveryId>:<attempt>`; the job re-reads consent. **Needs migration 0024** — applied locally, not yet on RDS.
- Email sends are **not** written to `notification_deliveries`, so the Send log's Email filter is empty.

15.9 Current status — it has never actually sent a message

As of 2026-10-14:

- Local `.env.local` has `WHATSAPP_ENABLED=true` but **no** `EUM_PHONE_NUMBER_ID`, so every WhatsApp records `skipped/disabled` and no AWS call is made. **Never commit** `WHATSAPP_ENABLED=true`**.**
- `notification_deliveries` had **no WhatsApp rows** — no message has been attempted end to end, and it has not been tested on a phone.
- Profiles with a number and consent: `prithish@sbcllp.in` (client), `abdulfarhath@sbcllp.in` (super admin).
- Settings has a **Send test message** button (`POST /api/account/whatsapp-test`) which sends **directly, bypassing Inngest**, with a 30 s cooldown, and explains skip reasons.
- Real notices go through Inngest: you must run `npm run inngest:dev` (port 8288) alongside `npm run dev`, or the enqueue throws and is logged as `[whatsapp] queue failed` with nothing recorded.
- `.env.local` changes need a `npm run dev` restart.

Diagnose with:

```
select status, skip_reason, error_code, created_at
from notification_deliveries
where channel = 'whatsapp'
order by created_at desc
limit 10;
```

15.10 The owner's remaining out-of-band setup

The code is **inert** until these are done (they are AWS/Meta console work, not code):

1. Under the existing verified Meta Business portfolio, create or attach the **transactional WABA + phone number** — separate from the DoubleTick number, which stays on its own integration for marketing. A number attaches to exactly one integration.
2. In the AWS End User Messaging (Social) console, link that WABA, register the number, and copy the **origination phone number id** into `EUM_PHONE_NUMBER_ID` (format `phone-number-id-0123…`).
3. Get the **fourteen templates approved** in WhatsApp Manager under names matching the event names, category **utility**, language `en`. Body variable order must match `notes/whatsapp-templates.md`.
4. Create the **SNS topic** EUM publishes to; subscribe the HTTPS endpoint `…/api/webhooks/aws-eum` (the route completes the handshake itself once deployed); set `AWS_EUM_SNS_TOPIC_ARN`.
5. Attach an IAM policy allowing `social-messaging:SendWhatsAppMessage` to the App Runner role. **No static keys.**
6. Confirm EUM Social is available in the chosen region (`SOCIAL_MESSAGING_REGION`, falling back to `AWS_REGION`), then set `WHATSAPP_ENABLED=true`.

Then: Settings → Send test message → run `inngest:dev` → as lead/manager approve or return a step → the client should get both the email and the WhatsApp.

Installation note: `@aws-sdk/client-socialmessaging` had to be installed with `--legacy-peer-deps` because of the pre-existing `eslint-plugin-jsx-a11y` / `eslint@10` peer conflict.

---

16. In-app notifications and announcements

Two separate systems that people constantly confuse:

	Notifications (the bell)	**Announcements** (the megaphone)
What	per-user work alerts	firm-wide news: Finance Act, circulars, process notes
Table	`notifications`	`announcements`
Who can post	the system, plus admin/manager for another user	super_admin, admin, manager
Who can read	the recipient only	every signed-in role

16.1 The bell

- It is **inbox-only**: the signed-in user's **received** rows. Not firm-wide, not sent email.
- It is a **Popover** (click outside or Esc closes it). Unread is `.unread-edge` — a 3 px left primary bar — **not a dot**.
- **Polling:** a 4-second **head** poll (`GET /api/notifications?head=1` → latestId / unread / count), and the full inbox query is invalidated **only when that fingerprint changes. Never put a `refetchInterval` on the full inbox.**
- **Live popup:** new ids pop up one at a time; the first visit seeds history so a refresh does not replay everything. Storage key `vcfo.notifications.popup.{userId}`. Closing (X / backdrop / Esc) "genies" into `[data-notifications-bell-target]` via the shared `measureGenieDock`; reduce-motion skips the flight; landing pulses the bell. **An auto-popup does not mark the row read.** No "Got it" button, no "parks on the bell" copy.
- Clicking a received row marks it read and reopens the card (`requestNotificationPopup` / `vcfo-notifications-show`). Clicking a *sent* row expands detail **inside the panel**.
- **Clear (today / this week / all) hides from the inbox** by setting `dismissed_at`. **Rows are never deleted.** Undo sets it back to null. A legacy `action: 'delete'` maps to dismiss. **Do not hard-delete from the bell.**
- "Today" = the calendar date in **Asia/Kolkata**. "This week" = Monday 00:00 IST through Sunday 23:59 IST, inclusive of today. Clear applies to the current Received/Sent tab.
- History pages: `/app/{role}/notifications`, served by `GET /api/notifications?history=1` (dismissed + current).
- The current bell UI is a plain bell with a red count badge, "N new", and Today / Yesterday / Earlier groups, with a footer link "Get these on WhatsApp" → Settings.

16.2 Direct-flow bell events (2026-10-13)

New events: `step_submitted`, `step_waiting_on_lead`, `step_reopened`. Recipients for these — and for the board resolution's `lead_requested_review` / `review_rejected` on `pre-2` and `board_resolution_shared` — come from **one pure table**: `directFlowBellAudience` in `engagement-event-fanout.ts`. The actor is excluded, and each person gets exactly one row.

Bell text: title `{Step} submitted`; body `{Name} ({Client | Project Lead}) submitted this step for {Company}.` The actor name travels in `NotifyInput.actorName` / `actorRole`, **not** a column. Reused kinds: `checklist.submit` (submitted, waiting on lead), `checklist.unlock` (reopened).

16.3 Announcements

- Routes `/app/{role}/announcements`. The navbar megaphone is the in-app list; the client inbox has a compact list too. `kind` + `author_role` on the row (migration 0010). **The author's name is stored on the row.**

- **The megaphone filter is All / Important / General, but there is no `important` kind**: Important = `deadline` + `compliance`; General = `general`; All = every kind.
- Live popup: new posts (whether composed or ingested from RSS) appear for every signed-in role within a few seconds, via a 4-second **head** poll (`GET /api/announcements?head=1`), refetching the full board only when the latest id or count changes. **Do not put a `refetchInterval` on the fat list — `JSON.parse` of announcement bodies on the main thread froze hover and clicks.**
- The card genies into `[data-announcements-bell]` (~1 s, Framer `m` + FLIP rects). Reduce-motion or appearance `motion === 'none'` skips the genie. **The genie does not mark read.** Already-shown ids live in `vcfo.announcements.popup.{userId}`; read state in `vcfo.announcements.read.{userId}`. Authors skip their own posts. The queue is one at a time, capped at 3 unseen on a first visit.
- **Official RSS/Atom only.** Staff paste a **feed URL** (not a homepage) from an allow-listed host (`OFFICIAL_FEED_HOSTS` in `src/lib/announcements.ts`). The Inngest `announcement-feeds` job runs once at **06:00 Asia/Kolkata**. Tracking junk (`utm_source=chatgpt.com`, `gclid`) is stripped from URLs.
- ⚠️ **We deliberately do NOT scrape HTML listing or login pages** (MCA, GST, EPFO portals, Income Tax "What's New"). If a department has no RSS feed, open the circular from the portals directory or post it by hand.
- Portals catalog: `src/lib/announcement-portals.ts`, rendered on the Announcements page. **LEI renewal uses `ccilindia-lei.co.in`** (the official LOU), not the ads agent at legalentityidentifier.in.
- `systemGetOrCreateAnnouncementSource` is only called **after a catalog URL actually parses as a feed**. Staff "Pull now" goes through `ingestAnnouncementSource`.

---

17. Ask VCFO — the in-app AI assistant

17.1 What it is and who sees it

An in-app assistant (⌘J) that answers a client's questions about their own incorporation, from **firm-reviewed content** plus **their live project state**.

- **Client, admin and super_admin only.** The mount is role-gated and every `/api/ask/*` route returns **403 for manager and lead**.
- **Off by default in code** (`ASK_VCFO_ENABLED=false`) — but **ON in production** since 2026-10-01 16:43 IST.
- Spec-by-spec status and how to turn it on: **`docs/specs/ask-vcfo/STATUS.md`**. Main spec: `docs/specs/ask-vcfo/ASK-VCFO-CONTEXT.md` v2.0. Phase-0 deviations and decisions: `PHASE-0-RECON.md`.

17.2 The pipeline

```
question
  -> Haiku GUARD                       (cheap classification / safety)
  -> a REVIEWED TOPIC, if one matches  (no LLM generation at all)
     else Sonnet `render_answer`       with read-only tools
  -> CODE VALIDATION of the output: dates, citations, visuals, step ids
  -> persist with token usage
```

Content lives in `src/data/ask/`: **11 topics** (plus glossary, suggestions, playbook) — and **all 25 topics are still marked DRAFT**, which means they stay hidden until a human reviews them. Pipeline code is in `src/lib/ask/` (`tools/`, `provider/`, `ingest/`).

Retrieval uses **Postgres full-text search over `ask_chunks` — there is no pgvector.** `ASK_VCFO_RETRIEVAL_TOP_K` defaults to 8.

17.3 The surfaces

- Panel `src/components/ask/` — push / overlay / bottom-sheet layouts, ⌘J.
- "What's this?" inline help and glossary underlining.
- **Client Library** (`src/views/client/ask/ClientLibrary.tsx`) plus a reader and a PDF brief. Currently **books only** (`ReferenceShelf`) — the owner has not sent the PDFs yet.
- **Saved answers** (the owner chose to keep these): the button is "Save answer"; the panel home lists the 5 newest "Saved answers"; each opens `/app/client/library/[id]` whose back link reads "Back to home".
- Super admin can **preview as client**.
- Admin page **Ask VCFO sources** for ingestion (via Inngest or inline).
- **Go-there links**: `destinations.ts` scope check, `resolve-href.ts`, an `?from=ask&focus=` arrival pulse, and a "Back to Ask VCFO" pill.
- "Ask VCFO about this" appears on each client-home card (`AskAboutThis`, hidden when Ask is off).

17.4 Feature flags (all currently OFF)

Flag	Feature
`ASK_VCFO_FEATURE_C1`	daily nudge (table `ask_client_prefs`, migration 0023)
`ASK_VCFO_FEATURE_C2`	explain a delivered document
`ASK_VCFO_FEATURE_C3`	"Why do we ask this?" field help (lines are drafts)
`ASK_VCFO_FEATURE_C4`	monthly status brief PDF
`ASK_VCFO_FEATURE_C5`	"What if I miss it?" on compliance items (obligation topics are drafts)
`ASK_VCFO_FEATURE_A1`	question gaps: `/app/admin/ask-vcfo/gaps`, groups client questions that had no reviewed topic. **Company names are shown to super admin only — that is decided in the repository.** While A1 is on, clients see the notice "Your firm can see questions you ask here, to help you better." (`ASK_VISIBILITY_NOTICE`; the owner may reword it.)

Terraform exposes these as `ask_vcfo_features = [...]`.

17.5 Configuration and models

Env	Default / note
`ASK_VCFO_ENABLED`	`false` in code; `true` in production
`ASK_VCFO_LLM_PROVIDER`	`anthropic` or `bedrock`
`ANTHROPIC_API_KEY`	server-only. **Empty locally means free-text answers say "unavailable" — suggestions and topics still work.** On AWS it is the Secrets Manager entry `/vcfo/ask-vcfo/anthropic-api-key`.
`ASK_VCFO_MODEL_GUARD`	`claude-haiku-4-5-20251001`
`ASK_VCFO_MODEL_ANSWER`	a Sonnet model

Env	Default / note
`ASK_VCFO_MODEL_CONTEXTUALIZER`	Haiku
`ASK_VCFO_ANSWER_EFFORT`	`low` / `medium` / `high`
`ASK_VCFO_RATE_LIMIT_HOUR` / `_DAY`	30 / 200
`ASK_VCFO_REFUSAL_FALLBACK`	`true`
`NEXT_PUBLIC_FIRM_DISPLAY_NAME`	`SBC` — used on "Reviewed by …" badges and PDF briefs

New dependencies this feature introduced: `@anthropic-ai/sdk`, `@anthropic-ai/bedrock-sdk`, `@react-pdf/renderer`.

17.6 Operational notes

- Deployed 2026-09-30 22:27 IST (`739dec1`), RDS migrated 0022 + 0023.
- Turned on in production 2026-10-01 16:43 IST: the owner created the secret and applied `ask_vcfo_enabled = true` (App Runner env `ASK_VCFO_ENABLED=true`, secret `ANTHROPIC_API_KEY`); a stale `WHATSAPP_PROVIDER` env var was removed at the same time.
- The first live eval scored **29/29**. The gap-closing build added a phase-progress tool, a "Where is my incorporation now?" answer, a topic review pack plus a publish command, and 19 Playwright e2e tests.
- ⚠️ `npm run eval:live` **has not been run since.** Doing so is a reasonable next task.
- Demo logins **do not work on production** — the only signed-in check on the live site is the owner's own.
- The **Ask VCFO bot face is deferred.** Ask VCFO behaviour should otherwise be left as-is unless asked.

---

18. The design system

18.1 Tokens (cool blue, professional)

- **Primary is professional blue**: `--primary` / `--brand` / `--blue-*` around `#2563EB`, hover `#1D4ED8`. Used for buttons, links, focus rings, current-step nodes, progress and key CTAs. White label on blue — **never navy on blue.**
- **Do not use orange, terracotta, peach, gold or sand** as a brand colour or page atmosphere. `--orange-*` and `--gold-*` are **deprecated aliases of** `--blue-*` so leftover class names still resolve blue; prefer `primary` / `blue-*` in new code.
- Neutrals are **cool slate with a blue undertone** (`--background` ≈ `#F8FAFC`, white cards, `--border` ≈ slate-200). **Never beige or cream.**
- **Phase washes** (journey chips only, ~8–12% chroma): `--phase-pre` sky, `--phase-filing` teal, `--phase-post` teal-green, `--phase-fema` indigo, `--phase-registration` violet-blue. Helper: `src/lib/phase-colors.ts` (`phaseKeyFromId`, `PHASE_CLASSES`).
- **Status colours appear in chips, icon tiles, dots and chart fills only — never as page or panel fill**: teal-green done, coral waiting, slate lock, rose/red overdue or error. **Never khaki, ochre or brown** — "waiting" used to be muted gold and read as brown; it is now coral.
- Categorical accents: `--accent-amber` is lemon, `--accent-orange` is tangerine (both high chroma).
- Metric top-bar colours use semantic tokens (`primary`, `danger`, `accent-sky`, `success`) — **never** `orange-*`, which still alias blue. Waiting is sky / pink / cyan.
- **Super Admin gets a tiny** `.super-gold-chip` **badge only — never a gold CTA theme.**
- **Dark mode stays in the same cool blue/slate family — not a brown invert.**

- Fonts: **Manrope** (UI), **Space Grotesk** (display, via `--font-serif`), **IBM Plex Mono** (identifiers).
- ⚠️ `--ink` **is a SURFACE token.** Never use `text-ink` for text — use `text-foreground` / `text-muted-foreground`.

18.2 The canonical component table

The **Project Lead dashboard is the canonical design system.** Where two versions of a primitive exist, the one the lead dashboard uses wins, and everything else is deprecated for new surfaces.

Element	Canonical component	Path
Panel / card container	the `.surface` utility (`@apply noir-card shadow-layered`), wrapped by `DashSection`	`app/globals.css`, `src/components/dash/DashSection.tsx`
Section header	`DashSection` header: a `h-7 w-7` solid tone tile + 11.5 px extrabold uppercase title	`src/components/dash/DashSection.tsx`
Status pill	`TONE_BADGE[tone]` on `rounded-full px-2.5 py-0.5 text-[11px] font-extrabold`; lead helper `internToneBadge(tone)`	`src/components/common/IconChip.tsx`, `src/components/intern/intern-tones.ts`
Progress bar	`LeadPhaseProgress` — `h-2.5` flex of `rounded-full bg-raised` segments	`src/components/intern/LeadPhaseProgress.tsx`
Progress ring	`DashHeroRing` (on a gradient hero) / `ProgressRing` (on a surface)	`src/components/dash/DashHero.tsx`, `src/components/noir/ProgressRing.tsx`
Greeting hero	`DashHero` **— the only one.** It absorbed the other two heroes; `LeadHero` and `ClientOverviewHero` are thin content wrappers	`src/components/dash/DashHero.tsx`
Data table	`DashDataTable`	`src/components/dash/DashDataTable.tsx`
Filter chip	`DashFilterChip`	`src/components/dash/DashFilterChip.tsx`
Charts	`src/components/charts/*` — themed tooltip, token-derived colour scale, default axis/grid props, lazy recharts loader	`src/components/charts/`
Donut	`DashDonut` + `DashLegendRow`	`src/components/dash/DashSection.tsx`
Journey node	`JourneyNode` (blue active pulse, teal check, coral clock, slate lock)	
Page header / back	`PageBackCluster` / `PageBackButton`, crumbs from `shell-crumbs.ts`	`src/components/shell/`
Page transition	`PageTransition` (wraps every view) + `SEO`	`src/components/shell/PageTransition.tsx`
Shell	`AppShell` → `RoleSidebar` + `TopBar` + `CommandPalette`	`src/components/shell/AppShell.tsx`
Motion presets	`cardHover`, `tweenShort`, `fadeUp`, `springSnappy`, `staggerKids`; `layoutId` pills via `MotionActivePill`	`src/lib/motion.ts`

Spacing / type scale: panel gap `gap-3`; panel padding `px-4 pt-3 pb-3`; radius `var(--radius)`; type scale 10.5 / 11 / 11.5 / 12 / 12.5 / 13 px with `font-extrabold` labels; serif display for numbers (`.serif`).

18.3 Composition rules

- **Page skeleton:** `PageTransition` → `SEO` → `div.flex.flex-col.gap-3` → hero → `grid xl:grid-cols-[minmax(0,1fr)_318px]` (main column + a fixed 318 px side rail).
- **KPI placement: headline numbers live INSIDE the hero panel**, not in a card band under it. A screen that shows the same numbers in the hero *and* in cards prints every number twice.
- **Panel anatomy:** `.surface` → header row `px-4 pt-3` with a `h-7 w-7` solid-tone icon tile and an uppercase 11.5 px title → body `px-4 pb-3 pt-2.5`. Use `DashSection` rather than hand-rolling it.

- **List rows:** `divide-y divide-border` or `border-t border-border`, row body `py-2.5`, 13 px semibold title, 11 px muted meta, status as a `rounded-full TONE_BADGE` pill, actions right-aligned as `rounded-full border border-border px-3 py-1 text-[11px] font-bold`.
- **Empty states:** in-panel = a one-liner at `text-[12.5px] text-muted-foreground`. Full-page = `.surface px-6 py-8 text-center` with a `serif` line. Illustrated variant: `EmptyStateIllustrated`.
- **Loading:** per-panel `animate-pulse rounded-md bg-muted/40` blocks — **never a full-screen boot screen.**
- **Callouts:** there is no shared callout component, and **do not add a second one.** Compose shadcn `Alert` + `IconChip tone="info"` with the login info tint.
- **App copy:** no instructional chrome. Do not add a helper line that restates a control (e.g. Sign out: "End this session on this device"). `PageHeader` `subtitle` is for live counts, dates and company names — not a second sentence of the H1. Empty states get a title only, or one short line. Keep `aria-label` / `sr-only`.

18.4 The shell

- Chrome is an **L flush to the viewport**: sidebar `top-0 left-0 bottom-0`, top bar `top-0` from the sidebar's right edge to the screen edge. No floating inset.
- ⚠️ `.shell-sidebar-skin` **must NOT set** `position`**.** Unlayered `globals.css` overrides Tailwind's `fixed` on the desktop `aside`, so the rail re-enters flow — icons land in a horizontal row at the top-left and the page drops below a full-height gap. This rule is repeated in **four** places in the notes because it keeps being re-broken.
- ⚠️ Do not add an opaque `::after` floor of `oklch(var(--panel))`. A negative z-index still paints *above* the host background, hiding `--sidebar-surface-bg`, so appearance skins look stuck on white.
- **Hover-peek** expands the collapsed rail over content (200 ms leave delay). Hover-peek state is **local to** `RoleSidebar`**, never** `ShellNavContext` — putting it in context re-rendered the whole workspace.
- The footer pin is **one** compact control (`SidebarPinButton`) cycling Auto (hover peek) → Keep open → Keep closed. **Do not restore the old dual Keep open / Keep closed buttons.** `sidebarCollapsed` is derived (`mode !== 'open'`).
- Disclosure rows (Clients, Docs, Updates) all share `SidebarNavDisclosure`. Chevron is Lucide `ChevronDown` (stroke, `fill-none`); the count badge sits immediately before it. The active pill/rail appears **only when that section's route is current.**
- **Top-bar trail** is the full location path from `shellBreadcrumb` (`shell-crumbs.ts`): `Home › Clients › DemoCo › SPICe+ Part B › Director KYC`. The separator is `›` (ChevronRight), **never ASCII** `->`. Every non-leaf crumb is a `Link`; the leaf is `aria-current`.
- `PageBackButton` sits beside the H1 on every AppShell page **except** the true home (lead Today, staff dashboards, client Inbox, role index). Click is `router.back()` when `history.length > 1`, else the parent path. The top bar has no back slot.
- `PageHeader` drops the giant H1 lockup when `pageTitleRepeatsTrail(title, pathname)`, keeping a visually hidden H1.
- Search is a **tool button**, not a fake input. `CommandPalette` is the only type-in search, and **only one may be mounted** (never a second in `AppShell`). Cmd/Ctrl+K focuses the TopBar input.
- The top bar always shows the official SBC lockup (`public/sbc-logo-light.png` / `sbc-logo-dark.png`, swapped by `html.dark`). Source exports were JPEG on an opaque black artboard; near-black was knocked to alpha so they do not sit as a black rectangle. `[data-role="admin"]` pins a light `--background` that beats `html.dark`, so login and marketing wrappers add `public-lockup-theme`.

18.5 Motion rules

- `LazyMotion` in `app/providers.tsx` must use `domMax`**, not** `domAnimation`. `m` + `layoutId` is a **silent no-op** without the layout feature — the sidebar pill simply never slides. Keep `strict` on.
- `LazyMotion strict` **throws** if a page renders `motion.*` instead of `m.*`. Prefer `import { m as motion }`.

- ⚠️ **Never put CSS `transform` / `transition-transform` / `active:scale` / Framer `whileTap` scale on a node that hosts or wraps a `layoutId` pill.** Projection uses transform, and a competing transform kills the shared-element spring. Press feedback there must be colour or opacity only.
- Do **not** import `_feature-registry` / `ui/_registry` from `providers.tsx` — that pulls the entire component catalog into the root client bundle and freezes the main thread.
- Always respect `useReducedMotion`.
- **AppContext's value identity must stay stable across the 4-second live poll.** Empty `internOptions` is a **module-level constant** — never `?? []` in render. Copy TanStack lists into reducer state only when item refs actually change.
- Never call `useClientLocaleNow()` in Today or AppContext — that re-rendered the whole lead home every second and froze hover and clicks. The quiet IST clock stays in `ClientLocaleNowLabel`, a leaf component.

18.6 Performance rules learned the hard way

- `listEngagements` does **not** select `checklist_state`.
- Tasks / requests / activity / invites / audit scope via `listScopedEngagementIds` (id-only).
- Dashboards hydrate from `GET /api/checklist-index` (answers stripped in SQL). Step pages and the vault still load the full `/checklist` per engagement.
- **Do not put a `refetchInterval` on the fat list.** Live popups stay on the 4-second `?head=1` poll.
- The command palette mounts the cmdk result list **only while open**.
- The compose-email host is **dynamically imported** from providers.
- Keep `useInternPortfolio` inside `InternMyWorkBadge` (memoised) so sidebar hover does not rebuild the lead queue.

---

19. Background jobs (Inngest)

Job	File	Trigger	Notes
Compliance generation	`src/jobs/compliance-generate.ts`	cron	Calls `systemGenerateComplianceInstances` / `runComplianceGenerate`. Upserts obligation rows on first run. **Not yet wired on App Runner** — run `npm run compliance:generate` by hand after creating engagements in production.
WhatsApp send	`src/jobs/whatsapp-send.ts`	event	Throttled 20/minute; dedupe id per event/person/day; retries thrown inside the `send` step.
Announcement feeds	`src/jobs/announcement-feeds.ts`	cron 06:00 Asia/Kolkata	Pulls allow-listed official RSS/Atom feeds.
Ask VCFO ingest	`src/jobs/ask-ingest.ts`	event	Chunks and indexes firm documents for Ask retrieval.

The Inngest client is `src/jobs/client.ts`; the HTTP handler is `app/api/inngest/route.ts`.

Locally you must run `npm run inngest:dev` (port 8288) alongside `npm run dev`, or any enqueue throws. Under `next dev` the SDK is in dev mode and posts to `localhost:8288`.

Inngest cloud keys for AWS are still deferred.

---

20. Testing and verification

20.1 What exists

Layer	Tool	Count
Unit + integration	**Vitest** (`vitest.config.mts` , jsdom)	**185 test files, ~1,560 tests**
Repository tests	**PGlite** (`@electric-sql/pglite`) — in-memory Postgres	the `*.test.ts` files inside `src/db/repositories/`
Component tests	`@testing-library/react` + `jest-dom`	across `src/components` and `src/views`
End-to-end	**Playwright** (`playwright.config.ts` , `e2e/`)	19 Ask VCFO e2e tests among others
AI evals	`npm run eval:live` (`tests/ask/evals/run-live.ts`)	first live eval scored 29/29

Tests are **colocated** with the code they cover (`src/lib/checklist-step-gate.ts` + `checklist-step-gate.test.ts`), which is the convention — follow it.

20.2 The gate, and the last recorded result

```
npm run typecheck
npx vitest run --testTimeout=60000
npm run lint
```

Last verified (2026-10-14, continuation session — recorded in `HANDOFF.md`):

- `typecheck` — **0 errors**
- full `vitest run --testTimeout=60000` — **171 files / 1,560 tests green**
- `npm run lint` — **0 errors / 171 warnings**

> **Use `--testTimeout=60000` .** The docx, zip and Ask-publish tests exceed the 5-second default **while `next dev` is running**, which it usually is on the owner's machine. Without the flag you will see false failures and waste time investigating them.

20.3 Typecheck traps

- `export interface Foo = Partial<Bar>` is **invalid**. Next overlays `Expected '{', got 'interface'` and `tsc` reports `TS1005` . Use `export type Foo = …` . That parse error in `AppContext.tsx` **500s every page**, because providers import it.
- `tsc` is incremental (`tsconfig.tsbuildinfo`). For a gate you fully trust, run with `--incremental false` — a stale build-info file once hid a real test typing error.
- If `tsc` reports missing modules (`checklist-step-attachments` , `InternStepDoneMark` , `InternSectionHeadingNav`) or extra props (`sectionTabs` , `extraFooterActions` , `hideStatus` , `sidebarMode`), the **consumers landed without the helpers**. Restore the helper and add the prop to the source type — **do not delete the call sites.**
- `resendManagerEmail` is a `ChecklistItemPatch` extra, **not** a persisted `ChecklistItemState` field.

20.4 Dead-code hygiene

`npx knip --reporter compact` is the reference scan. Expected residue: `src/test/server-only-stub.ts` only. Anything else is new dead code.

A sweep on 2026-09-13 removed ~100 unreachable files, dropped 22 unused npm dependencies, deleted the Supabase-era shims, removed the no-op localStorage stubs (`src/lib/storage.ts` , `regenerateComplianceForEngagement`) and the `mockData` seeds, and deleted or un-exported ~180 dead exports and types. `GoldButton` was migrated to `AccentButton` .

> **Never add keep-alive "registry" files that import unused modules to silence tooling.** That trick hid roughly 60 dead files before the sweep.

---

21. Deployment

21.1 The three ways this app runs

Mode	Command	Who it is for
Local dev	`npm run dev`	the owner's daily work (localhost:3000)
Office LAN pilot	`npm run dev:lan`, or `npm run build && npm run start:lan`	~10 people on office WiFi; data never leaves the building
AWS production	push `:latest` to ECR; App Runner auto-deploys	the live product

21.2 Current AWS production

Deployed since 2026-09-11, region **ap-south-1** (Mumbai), account **600627321277**.

Resource	Value
App	**App Runner** service `vcfo-suite` — auto-deploys on `:latest`
Custom domain	`app.sbctrack.in` (ACM cert auto-issued; GoDaddy CNAME — an apex domain is not possible without ALIAS)
Original URL	`https://ebuwqhvnuy.ap-south-1.awsapprunner.com`
Database	**RDS** `vcfo-suite-db`, Postgres 16, `db.t4g.micro`, **public** + `rds.force_ssl` (network posture "option B")
Storage	**S3** `vcfo-suite-documents-600627321277`
Registry	**ECR** `vcfo-suite`
Secrets	Secrets Manager `vcfo-suite/{DATABASE_URL, AUTH_SECRET, AZURE_AD_CLIENT_SECRET}` and `/vcfo/ask-vcfo/anthropic-api-key`
Email	**SES** identity `sbctrack.in` — DKIM CNAMEs pending at DNS; account still in the **SES sandbox** (quota case 178914709000400 opened 2026-09-11)
Budget	**$25**
IaC	Terraform in `infra/` with remote state in `vcfo-suite-tfstate-600627321277`. Runbook: `infra/README.md`

Terraform files: `main.tf`, `network.tf`, `database.tf`, `storage.tf`, `app.tf`, `ses.tf`, `budget.tf`, `variables.tf`, `outputs.tf`.

21.3 AWS deploy gotchas (each of these cost real time)

1. `aws login` **sessions are invisible to Terraform.** The CLI's browser login writes a `login_session` the Go SDK cannot read. Run `eval "$(aws configure export-credentials --profile vcfo --format env)"` **immediately before every terraform command**. The exported token expires in ~20 minutes, so a long apply started on a stale export dies mid-run, leaving an `errored.tfstate` and a held S3 lock — recover with `terraform force-unlock <id>` using the id from the `.tflock` object.
2. **App Runner injects** `HOSTNAME=<instance>`, overriding the Dockerfile ENV. Next standalone then binds to it and the health check never reaches port 3000 (`CREATE_FAILED`). The CMD forces `HOSTNAME=0.0.0.0` at exec time. **A** `CREATE_FAILED` **service cannot be redeployed — you must delete and recreate it, and the service URL changes.**

3. `pg` ≥ **8.16 treats** `sslmode=require` **as verify-full.** RDS chains to Amazon's private CA, so without `NODE_EXTRA_CA_CERTS=certs/rds-global-bundle.pem` every connection fails with "self-signed certificate in certificate chain". An explicit `ssl: { rejectUnauthorized: false }` does **not** override the URL. **This applies to running** `db:migrate` **from your laptop too.**
4. `npm ci` **needs** `legacy-peer-deps=true` (in `.npmrc`) because of eslint 10 vs eslint-plugin-jsx-a11y. `drizzle-kit` had to move 0.18.1 → 0.31.x: the old version had no `defineConfig`, which only `next build` typechecks.
5. `terraform -exclude` **does not exist in 1.13.** The bootstrap order uses `-target` for everything except the App Runner service, which needs an image to exist first.
6. `NEXT_PUBLIC_SITE_URL` **is baked at build time.** Server-side links read the runtime `SITE_URL` first (`src/lib/site-url.ts`), so **one image can serve any hostname**.

21.4 Deploying a change

```
# 1. verify locally
npm run typecheck && npx vitest run --testTimeout=60000

# 2. build + push the image to ECR (see infra/README.md / buildspec.yml)
#    App Runner auto-deploys on :latest

# 3. if the change includes a migration, run it against RDS from a trusted machine:
export DATABASE_URL="$(aws secretsmanager get-secret-value --profile vcfo \
  --secret-id vcfo-suite/DATABASE_URL --query SecretString --output text)"
NODE_EXTRA_CA_CERTS=certs/rds-global-bundle.pem npm run db:migrate
```

Recorded deploys: `eaddef0` (2026-09-15, RDS migrated 0019), `6954696` (2026-09-16), `96cfafa` (2026-09-15), `739dec1` (2026-09-30, RDS migrated 0022 + 0023), `49e8386` (2026-10-13, no migration).

⚠️ **RDS is currently migrated up to 0023. Migration 0024 is NOT applied.**

21.5 Explicitly deferred AWS work

- **SES production access** (out of sandbox) + **DKIM verification** (records are in DNS, pending)
- **Private RDS / VPC connector** (network posture "option A")
- **Inngest cloud keys** — so the compliance cron does not run on AWS; run `npm run compliance:generate` manually
- `project` cost-allocation tag activation (the key is not yet visible in Billing)
- **Change or purge the seeded demo passwords on RDS before real users.** This is the single most important item on the list.
- Email password-reset links (today the forgot-password UX tells the user to ask their manager)

21.6 Cost sanity

A quiet 10-user pilot is roughly **$15–40/month** (micro RDS + a small container + S3 + light egress), **before** WhatsApp message fees. Credits may cover the first weeks; they will not cover it forever. A **$25 budget alert** is already set. Prefer `db.t4g.micro`, single-AZ. Do not enable unused services.

21.7 Go / no-go guidance from the deploy doc

> **Go to AWS when** the local/LAN pilot is boringly reliable and you need offsite access. **Stay local when** you are still changing KYC/assignment flows daily — deploy churn will slow you down more than hosting helps.

---

22. Environment variable reference

Copy `.env.example` to `.env.local`. The file is heavily commented — read it.

22.1 Core

Variable	Local	AWS
`DATABASE_URL`	`postgresql://vcfo:vcfo_local_dev@localhost:5432/vcfo`	RDS URL + `?sslmode=require`, from Secrets Manager; the process must trust `certs/rds-global-bundle.pem` via `NODE_EXTRA_CA_CERTS`
`AUTH_SECRET`	`openssl rand -base64 32`	a **new** random value
`AUTH_URL`	**leave unset**	`https://app.sbctrack.in`
`AUTH_TRUST_HOST`	`true`	`true`
`NEXT_PUBLIC_SITE_URL`	`http://localhost:3000`	the same public HTTPS URL as `AUTH_URL`
`ALLOWED_DEV_ORIGINS` / `DEV_TUNNEL_HOST`	optional, for LAN / tunnels	n/a

22.2 Storage

Variable	Local (MinIO)	AWS
`S3_ENDPOINT`	`http://localhost:9000`	**unset / delete**
`S3_REGION`	`us-east-1`	`ap-south-1`
`S3_BUCKET`	`vcfo-documents`	the real bucket name
`S3_ACCESS_KEY_ID` / `S3_SECRET_ACCESS_KEY`	`vcfo_minio` / `vcfo_minio_local_dev`	**omit** if using an IAM role
`S3_FORCE_PATH_STYLE`	`true` (required for MinIO)	`false` **or unset**
`NEXT_PUBLIC_MAX_UPLOAD_MB`	`50`	same

22.3 Email

Variable	Note
`EMAIL_PROVIDER`	`resend` (default) or `ses`
`EMAIL_FROM` / `RESEND_FROM_EMAIL`	must be on a **verified domain**. Production: `VCFO Suite <info@sbctrack.in>`
`RESEND_API_KEY`	**empty locally = emails log to console**
`EMAIL_REPLY_TO`, `EMAIL_PROGRESS_CC`	optional (legacy `RESEND_*` aliases work)
`EMAIL_DEV_REDIRECT_TO` / `RESEND_DEV_REDIRECT_TO`	rewrites every To/Cc for local testing. **Must be unset in production. Never set it to the client address** — submit mail must reach the lead
`SES_REGION`, `SES_FROM_EMAIL`	only when `EMAIL_PROVIDER=ses`
`AZURE_AD_CLIENT_ID` / `_SECRET` / `_TENANT_ID`	Microsoft Graph for staff compose

22.4 WhatsApp

Variable	Note
`WHATSAPP_ENABLED`	`false` by default; **never commit** `true`
`SOCIAL_MESSAGING_REGION`	falls back to `AWS_REGION`

Variable	Note
`EUM_PHONE_NUMBER_ID`	**not set yet — this is what makes the feature inert**
`WHATSAPP_TEMPLATE_LANG`	`en`
`META_API_VERSION`	`v21.0`
`AWS_EUM_SNS_WEBHOOK_URL`	`https://your-host/api/webhooks/aws-eum`
`AWS_EUM_SNS_TOPIC_ARN`	optional but **recommended** — pins the topic
`WHATSAPP_TEMPLATE_NAME_<EVENT>`	overrides only; names default to the event name

22.5 Ask VCFO

See section 17.5. Key ones: `ASK_VCFO_ENABLED`, `ASK_VCFO_LLM_PROVIDER`, `ANTHROPIC_API_KEY`, the three `ASK_VCFO_MODEL_*`, the rate limits, and the six `ASK_VCFO_FEATURE_*` flags.

22.6 Other

Variable	Note
`NEXT_PUBLIC_SUPPORT_PHONE_E164`	when set, the client sidebar's "VCFO support" row gains WhatsApp and Call buttons
`NEXT_PUBLIC_FIRM_DISPLAY_NAME`	`SBC`
`NEXT_PUBLIC_ENABLE_DEMO_AUTH`	⚠️ **a password-less "sign in as client" shortcut.** Anything other than `"true"` disables it. **Must never be `true` on a machine other people can reach, and must be unset in production.**

---

23. The owner and how they work

This section matters as much as the architecture. Many past sessions created rework by ignoring it.

23.1 Working style

- **Owner:** Prithish (SBC / VCFO Suite).
- **Tests on localhost only**, logged in as the **Routeware client** (`prithish@sbcllp.in`), and **sends screenshots of what to change.** Messages are short. **Read the screenshot carefully** — it usually contains the whole request.
- ⚠️ **Do not commit or push unless asked.** A large amount of the current work is intentionally uncommitted.
- ⚠️ **Do not start a second `next dev`.** The owner runs `npm run dev` on :3000 themselves.
- From the user's own stored preference: **teach, don't just do.** Give full A-to-Z explanations in clear English; make sure understanding comes before the work; do not hide steps or reasoning.

23.2 The product goal for the client portal

> A professional, institutional CA/finance product that a client understands at a glance. Tight margins, no empty space, colour only where it identifies something, **every button works.**

23.3 Feedback already given — do not regress these

- **No dropdowns for phase steps** — they open **in place, on screen**.
- **Every opened view must name where it goes back to** ("Back to home", "Back to phases").
- **Status colours:** completed **green**, your turn **blue** (primary), with the team **amber**, upcoming **grey**.
- **No content expanding beside a narrow column**, and no `overflow-hidden` clipping.

- `--ink` **is a SURFACE token** — never `text-ink` for text; use `text-foreground` / `text-muted-foreground`.
- **Dark mode must stay readable** — `.dark [data-role="client"]` overrides the pinned light surface.
- **Floating buttons must not cover dashboard content.**

23.4 Keep as-is unless explicitly asked

- the compliance calendar page
- `ClientTimeline`
- the VCFO-by-SBC logo
- Ask VCFO behaviour (the bot face is deferred)
- **Do not copy the packed HTML prototype's look.**

23.5 The v7 prototype — a functional reference only

```
C:\Users\Prithish Sai Reddy\Downloads\vcfo-client-portal-prototype-v7-preview.html
```

An 11 MB single file. **Use it for *what each screen lets the client do*, not for how it looks.** The screens are JSON inside `<script type="application/json" id="pf">` → `.text["s/home/v7c_*.dc.html"]`; `routes.js` in the same map lists every screen; each component's behaviour and sample data sit in its `<script type="text/x-dc">` block.

What it defines: Home = greeting (+ Report), **Needs from you** (Upload / Fill / Correct / Sign tabs), **Who holds it now** (You / SBC / MCA), **Important alerts** (15 / 30 days, Fix), **What SBC is doing**, **Timeline**, **Steps by phase**; sidebar team rows carrying WhatsApp / Call / Email / Teams; nav showing counts (Updates 3, Incorporation 6/14, Documents 6).

---

24. The gotchas index

Twelve files in `docs/context/notes/` hold the hard-won knowledge. Read the one that matches what you are about to touch.

File	Covers
`data-access.md`	four-role scoping, multi-client, Super Admin, soft delete, change requests, Knowledge Bank folders, avatars, Phase-2 status mappings, runtime performance
`checklist-workflow.md`	the sequential gate, step visibility, Dependent/Independent, Part A + NIC + windows, the Part B restructure, the pre-inc direct flow
`lead-ui.md`	the Project Lead workspace: overview, step workspace, autosave, error summary, typecheck seams, Today + My work, vault, sidebar order
`shell-ui.md`	sidebar skin vs `fixed`, disclosure rows, the location trail, titles vs crumbs, motion, app copy, SBC lockups
`design-system.md`	tokens, the canonical component table, composition rules, known duplicate primitives
`compliance.md`	pre-incorporation visibility, the Compliances nav, the statutory calendar, the two colour lanes
`email.md`	the dispatcher, From/Reply-To rules, the process fan-out, Outlook Graph, the compose page, autosave vs approval email, direct-flow mail
`notifications.md`	the live popup, dismiss/history, announcements, RSS policy, direct-flow bell events
`incorporation-docs.md`	the document pack, per-director slots, DIR-8 row loops, template build scripts, the board-resolution variant
`aws.md`	AWS deploy gotchas, the whole WhatsApp/EUM design, the out-of-band Meta setup, local WhatsApp testing
`tooling.md`	environment facts, typecheck traps, LAN/phone access, dead-code hygiene

File	Covers
`whatsapp-templates.md`	the fourteen template bodies and their variable order

24.1 The twenty most expensive gotchas, in one list

If you read nothing else, read these.

1. **Docker Desktop must be running** before `npm run dev`, or login fails with `ECONNREFUSED`.
2. **Use `--testTimeout=60000`** for Vitest while `next dev` is running, or docx/zip/Ask tests fail falsely.
3. `proxy.ts` **must import** `@/auth/edge`, never `@/auth/config` — otherwise bcrypt and the pg pool load on every guarded request.
4. `.shell-sidebar-skin` **must not set** `position` — it breaks the whole shell layout. (Repeated four times in the notes.)
5. `LazyMotion` **must use** `domMax`, and never put a `transform` on a node hosting a `layoutId` pill.
6. `patchChecklistItem` **must merge onto the persisted row** and ignore the browser's `current` copy, which is role-redacted.
7. **Draft = saved ⊕ touched.** A full snapshot taken before the checklist loads sends `''` for every server-held field — a complete wipe.
8. **Do not key the autosave flush effect on callbacks** — the cleanup then runs every render and cancels the debounce.
9. **Autosave must not re-fire the manager-approval email** — notify only on `reviewSource=lead_manager_request` or `resendManagerEmail`.
10. **Never add a route returning the raw** `checklist_state`**.** Always `checklistStateForViewer`.
11. `isChecklistStepSequentiallyComplete` **takes an optional** `itemId` **— pass it**, or direct-flow steps read with the old rules.
12. **Removing a repeat entry must write** `''` **to its keys**; never key anything by entry index.
13. `pg` **≥ 8.16 needs** `NODE_EXTRA_CA_CERTS` for RDS, locally and in the container.
14. **App Runner overrides** `HOSTNAME`; a `CREATE_FAILED` service must be deleted and recreated, which changes the URL.
15. **Export Terraform credentials immediately before every command**; the token lives ~20 minutes.
16. **Leave** `AUTH_URL` **unset** for LAN and tunnel access.
17. **Next 16 blocks cross-origin** `/_next/*` — the page loads but controls are dead. Check for 403s on chunks.
18. **No** `refetchInterval` **on fat lists** (the engagement list, the full notification inbox, the announcement board). Head-poll at 4 s instead.
19. `export interface Foo = …` **is invalid TypeScript** and 500s every page from `AppContext.tsx`.
20. **WhatsApp retries must throw inside the** `send` **step** — a step that returns is memoised, so throwing afterwards only replays the stored failure.

---

25. Current state — what is done, what is uncommitted, what is deferred

The authoritative, dated version of this is `docs/context/STATE.md`. Summary as of 2026-10-16:

25.1 Done and verified

Setup / seed / Auth.js login · all Phase-1 API ports (the `_reference-supabase/` folder is deleted; ~35 real routes became ~120) · all listed repositories · Phase-2 localStorage → API (AppContext on TanStack Query) · Phase-3 compliance Inngest job + the shared email helper · process-lifecycle emails and multi-user notifications · multi-client per project + client self-invite · client-side audit trail · the Super Admin bird's-eye role · `npm run start:lan` · light/dark theme · create project +

client · change password · the five-role model · compliance visible pre-COI · the SPICe+ Part B restructure · Part A intake + NIC + MCA tab order + manager windows + the lead compliance gate · Dependent/Independent company type · Outlook on AWS · manager-gated delivery · step visibility per viewer · manager approvals from step / dashboard / bell · the staff project detail = client incorporation view · the Compliances nav group for every role · the dead-code sweep · **AWS Stage 2 deployed** · the pre-incorporation direct flow · Ask VCFO (on in production) · WhatsApp LCC parity (built, never sent) · incorporation drafts per director · fully sourced incorporation docs · Part B restructure (Document Execution removed, KYC/DSC stripped).

25.2 Uncommitted work in the tree right now

Branch `client-dashboard-v7`, on top of commit `97dabd6`:

```
 M app/globals.css
 M src/views/admin/FirmDashboard.tsx
 M src/views/client/Overview.test.tsx
 M src/views/client/Overview.tsx
?? src/components/client/orbit/        (17 files)
?? src/lib/orbit/                      (9 files)
?? scratch-flows.json / scratch-flows.txt / scratch-rich.json
```

The client portal redesign (October 2026) — done, uncommitted, owner testing:

- **Home** `src/views/client/Overview.tsx` is one sheet: navy-gradient hero (`.client-hero`) with an animated `ProgressRing`, a Report PDF download (`app/api/client/report`), Ask VCFO, and four hued KPI tiles (`.client-kpi`, hue via `--kpi`) → **Needs from you** (`ClientNeeds`: Upload / Fill / Correct / Sign tabs from `buildNeeds`; the due date is the manager's step window `to`) → four phase cards (carrying their phase hue from `PHASE_HUE` in `ClientPhaseBars`) → three columns: **Who holds it now** (`ClientWhoHolds`: You / VCFO team / MCA via `stepHolder`; MCA = `pre-5`, `pre-11`, `pre-12`), **Important alerts** (`ClientAlerts` + `src/lib/client-alerts.ts`: dated client steps + unfiled filings, 15/30 days, overdue always shown; the overview repo returns unfiled filings up to 60 days overdue as `status: 'overdue'`), and Recent activity (a violet rail).
- Sign steps are `pre-3`, `pre-8`, `pre-9` (`SIGN_STEPS`). Needs rows are coloured by action: upload sky, fill primary, correct danger, sign violet. Every card title is `CardHeading` with an icon tile in the card's hue; `CardTabs` are pills. Entrance (`.client-rise`) and pulse (`.client-live`) animations are off under `prefers-reduced-motion`.
- **Deleted:** `ClientNextAction`, `ClientBallInCourt`, `ClientAhead`.
- **"Your step right now"** (`src/components/shell/YourStepNotice.tsx`): a bottom-right side card (above the floating Ask button), **not** a full-width banner. Hides itself after 12 s unless hovered/focused. Once per step per session (`sessionStorage['vcfo.your-step']`). Shown on the client home, lead Today, and the manager dashboard.
- **Client sidebar:** the mini compliance calendar is replaced by **"Your team"** (`SidebarClientTeam` → `ClientTeamCard`) with brand-coloured WhatsApp (always), Call (only with a profile phone), Email and Teams. WhatsApp opens `wa.me/<their number>`, else `wa.me/<NEXT_PUBLIC_SUPPORT_PHONE_E164>`, else `wa.me/?text=…` (WhatsApp's own contact picker) — always with a prefilled message. A "VCFO support" row uses `info@vcfosuite.com`. Nav counts (client only, `ClientNavMeta.tsx`): Updates = unread notifications, Incorporation = done/total, Documents = open upload steps. **Staff sidebars still show the calendar.**
- **Incorporation page** uses the same `ClientPhaseBars` (from `buildProgress` / `buildMilestones`) with the schedule window and "What's this?" slots. The lead's `InternPhaseEntryCards` is untouched.
- **Board resolution:** client `pre-2` / `pre-3` show the document on a stage with approval actions under it; the client `/board-resolution` page and the lead editor were restyled with paper margins.
- **Team page** `src/views/client/ClientTeam.tsx`: Your VCFO team, People from your company (Replace = the old Substitute), inline "Give someone else access".
- **Filings** `src/views/compliances/FilingsView.tsx` (shared by every role): Back to home, plus a glance strip (Overdue / Due in 14 days / Upcoming / Filed for the whole FY + "Next filing" linking to the calendar date), helper

`summariseFilings` in `compliance-scope.ts`.

- **Documents** `src/views/client/Documents.tsx`: Back to home, a 4-box summary, "Still needed from you" first, files on record as aligned rows (name · section · date · file) grouped by milestone, answered requests, then the board-resolution card.
- **Timeline** (`TimelineChart.tsx`, used by `ClientTimeline` and staff `ProjectTimeline`): laid out in px over `timelineWorld(model)` — whole months, 2 before and 3 after the phases. Drag / swipe / shift+scroll / arrow keys to pan; ‹ Today › buttons; − / + / Fit and ctrl+scroll zoom around the cursor; a ruler with month labels plus Days/Weeks/Months ticks. Opens on the old fit view. The "Request a change" editor still uses the fixed axis.
- **Notification bell** restyled (see 16.1).

`src/components/client/orbit/` **(17 files) +** `src/lib/orbit/` **(9 files)** are **the new UI direction** (owner, 2026-10-16). This is not a stray experiment — it is the **replacement for the existing dashboards**, starting with the client portal: an "orbit map", a "dates deck", an "ascent path" and a "status brief", each with its own tests (`orbit-model.ts`, `orbit-geometry.ts`, `orbit-selectors.ts`, `orbit-palette.ts`, `orbit-area.ts`, `orbit-today.ts`, plus `OrbitMap`, `PortfolioOrbit`, `AscentPath`, `DatesDeck`, `DetailDrawer`, `DockPanel`, `StatusBrief`).

This is the active work. Continuing it is the job. The v7 client-portal redesign described above is the step before it; orbit is where the dashboards are going next. The three `scratch-*.json` / `.txt` files at the repo root are scratch output from that work and are not part of the product.

25.3 Intentionally deferred

- **Change or purge the seeded demo passwords on RDS before real users** (the most important one)
- Email password-reset links (forgot-password currently tells the user to ask their manager)
- **Cross-tenant repository tests** (the owner has been testing manually)
- AWS: SES production access + DKIM verification; private RDS/VPC connector; Inngest cloud keys; cost-allocation tag activation
- The **full email digest from the compliance job** (console log only today)
- Operational Readiness as primary navigation/queues (the data is kept but filtered out of the primary UX)
- Library PDFs (the owner will send them)
- The **Ask VCFO bot face**
- **Migration 0024 on RDS**
- `npm run eval:live` has not been re-run since the first 29/29

25.4 Open questions / unreproduced issues

- **Lead 404 on Subscriber Details (`pre-16`)** — reported by the owner, **not reproducible on 2026-10-14** (the step routes exist, the slug `rootware` is valid, no 404 in the dev log). Waiting for the owner to retry and send the URL if it still happens.

---

26. How to continue — the onboarding sequence for a new instance

If you are a fresh Claude Code instance picking this up, do this in order.

Step 1 — Orient (do not write code yet)

```
cat CLAUDE.md
cat docs/context/STATE.md
cat docs/context/HANDOFF.md
ls docs/context/notes/
```

Read this knowledge-transfer document. Then read the notes file for whatever area the owner's first request touches.

Step 2 — Get it running and prove it works

```
# Docker Desktop must already be running
npm install
npm run infra:up
npm run db:migrate
npm run db:seed
npm run dev
```

Log in as each of the five demo roles and click around. **Do this before changing anything** — it builds the mental map far faster than reading code, and it proves your environment is sane.

Step 3 — Establish your own verified baseline

```
npx tsc --noEmit -p .
npx vitest run --testTimeout=60000
npm run lint
```

Write down the numbers. If they do not match section 20.2 (0 errors, ~1,560 tests green, 0 lint errors / ~171 warnings), **find out why before you build on top of it.** The uncommitted work in the tree may shift the test count — that is expected; a *failure* is not.

Step 4 — Pick up the orbit UI work

The tree has uncommitted changes and two untracked folders (`orbit/`). **That orbit code is the active job**: it is the new UI replacing the existing dashboards, starting with the client portal (section 25.2). Read those 26 files and their tests before writing anything new, then continue from where they stop.

Do not commit, push, or delete any of it without being asked. The v7 redesign underneath it is still under the owner's review.

Step 5 — Work the way this project works

For each change:

1. Read the relevant notes file first.
2. Find the **pure module** that owns the rule (there usually is one — e.g. `pre-inc-direct-flow.ts`, `part-a-sections.ts`, `checklist-visibility.ts`). Change the policy in that one place rather than branching at the call sites.
3. If it touches data, add or change a **repository function** — never import `db` elsewhere.
4. Add or update the **colocated test**.
5. Run the gate (typecheck + tests + lint).
6. **Update** `docs/context/STATE.md` **and** `HANDOFF.md`, and append any new gotcha to the right `notes/` file.
7. Report honestly what is green and what is not.

Step 6 — Candidate next tasks (not yet assigned)

Ordered roughly by value. **Confirm with the owner before starting any of them** — the owner drives priority from screenshots, not from backlogs.

Priority	Task
THE JOB	**Continue the orbit UI** — the new dashboards replacing the existing ones, starting with the client portal (sections 25.2 and 26 Step 4).

Priority	Task
High	Change or purge the demo passwords on RDS. This is a live security exposure.
High	Apply migration 0024 to RDS so the WhatsApp Send-log retry works in production.
High	Finish the WhatsApp out-of-band setup (section 15.10) and do the first real end-to-end test on a phone.

| **Medium** | Commit the client portal redesign once the owner signs off. | | **Medium** | Write the deferred **cross-tenant repository tests**. This is the biggest correctness gap in the codebase. | | **Medium** | Review and publish the 25 draft Ask VCFO topics (`docs/specs/ask-vcfo/TOPIC-REVIEW.md` , `npm run ask:topics:publish`). | | **Medium** | Re-run `npm run eval:live` for Ask VCFO. | | **Lower** | SES production access + DKIM so email leaves the sandbox. | | **Lower** | Fix the `super_admin` fall-through in `compliance.ts` (section 8.5). | | **Lower** | Wire the compliance digest email (it only console-logs today). | | **Lower** | Give repeat entries stable ids so director renumbering stops re-keying drafts (section 11.2). | | **Lower** | Wire Inngest cloud keys so the compliance cron runs on AWS. |

Appendix A — Command cheat sheet

```
# ---- infrastructure ----
npm run infra:up            # start Docker Postgres + MinIO (idempotent)
npm run infra:down          # stop them
docker exec vcfo-postgres psql -U vcfo -d vcfo     # psql (user is vcfo!)

# ---- database ----
npm run db:generate         # generate a migration from schema.ts
npm run db:migrate          # apply migrations
npm run db:push             # push schema without a migration (dev only)
npm run db:studio           # Drizzle Studio
npm run db:seed             # demo users + engagement
npm run db:seed-demo        # richer demo firm
npm run db:purge-demo       # remove demo data

# ---- running ----
npm run dev                 # localhost:3000
npm run dev:lan             # 0.0.0.0:3000 (office WiFi)
npm run build && npm run start:lan
npm run inngest:dev         # Inngest dev server on :8288

# ---- verification ----
npm run typecheck
npm run test
npx vitest run --testTimeout=60000       # use this while next dev runs
npx vitest run <file>                    # one file
npm run lint
npx knip --reporter compact
npm run e2e                              # Playwright

# ---- jobs / content ----
npm run compliance:generate
npm run ask:topics:report
npm run ask:topics:publish
npm run eval:live

# ---- AWS ----
eval "$(aws configure export-credentials --profile vcfo --format env)"
export DATABASE_URL="$(aws secretsmanager get-secret-value --profile vcfo \
  --secret-id vcfo-suite/DATABASE_URL --query SecretString --output text)"
NODE_EXTRA_CA_CERTS=certs/rds-global-bundle.pem npm run db:migrate
```

Useful SQL

```
-- WhatsApp delivery diagnosis
select status, skip_reason, error_code, created_at
from notification_deliveries
where channel = 'whatsapp'
order by created_at desc limit 10;

-- one engagement's checklist status without the huge blob
select id, slug, stage, ownership_type, incorporation_date, deleted_at
from engagements order by created_at desc limit 20;

-- who is who
select email, role, intern_id, client_id, status from profiles order by role;
```

Appendix B — Glossary (Indian incorporation and compliance)

Essential for reading the checklist, the generated documents, and the client's questions.

Term	Meaning
MCA	Ministry of Corporate Affairs — the company registrar.
SPICe+	MCA's integrated incorporation application. **Part A** reserves the name; **Part B** does the actual incorporation. The checklist's Phase 1 and Phase 2 mirror them.
COI	Certificate of Incorporation — the moment the company legally exists. Step `pre-12`. Everything compliance-related starts here.
CIN	Corporate Identity Number — the company's MCA id, issued with the COI.
MOA (INC-33)	Memorandum of Association — the company's objects, state, and share capital clauses.
AOA (INC-34)	Articles of Association — the company's internal rules.
AGILE-PRO-S (INC-35)	The combined form for GSTIN, EPFO, ESIC, Profession Tax and bank account at incorporation.
DIN	Director Identification Number.
DSC	Digital Signature Certificate — required to sign MCA filings.
DIR-2	A director's consent to act as director.
DIR-8	A director's declaration of disqualification/prior directorships.
INC-9	Declaration by the first subscribers and directors.
NOC	No Objection Certificate — e.g. from a trademark owner for name use, or from a property owner for the registered office.
Board resolution	The parent company's formal decision authorising the Indian subsidiary. Step `pre-2`; signed copy `pre-3`.
Subscriber	A person or body corporate taking the company's initial shares. Step `pre-16`.
Registered office	The company's official address. Step `pre-14`.
NIC code (NIC-2008)	The 5-digit national industrial classification of the business activity. 1,302 sub-classes in `nic-2008.json`. Required in `pre-1` and printed in the board resolution.
PAN / TAN	Permanent Account Number (tax id) / Tax Deduction and Collection Account Number.
GST / GSTIN / GSTR-3B	Goods and Services Tax, its registration number, and the monthly return.

Term	Meaning
TDS	Tax Deducted at Source.
FEMA	Foreign Exchange Management Act — governs foreign investment. A whole checklist bucket.
FDI reporting	Reporting foreign investment to the RBI (FC-GPR and similar), a Phase-3 task.
EPFO / ESIC	Employees' Provident Fund Organisation / Employees' State Insurance Corporation.
DGFT / IEC	Directorate General of Foreign Trade / Import-Export Code.
Shops & Establishments	State-level registration for a place of business.
Professional Tax	A state-level employment tax.
LEI	Legal Entity Identifier. Renewal goes through `ccilindia-lei.co.in`, the official LOU.
FY	Financial year — in India, 1 April to 31 March. "FY 2026-27" is the demo register's year.
IST	Asia/Kolkata. **All "today" / "this week" logic in this app is IST**, never UTC. Dates go through `ymdFromIsoInIst` — never `slice(0, 10)` on a UTC timestamp.
GCC	Global Capability Centre — the kind of subsidiary most of these clients are setting up.
Dependent / Independent	This app's terms for "has a parent entity" vs "standalone". See section 1.4.
Engagement	The code/database word for one company's project. The UI calls it a **project**.
Project Lead	The product name for the code role `intern`.

Appendix C — Document provenance

This document was compiled on **2026-10-16** by reading, in the repository itself:

`CLAUDE.md` · `README.md` · `docs/context/STATE.md` · `docs/context/HANDOFF.md` · all twelve `docs/context/notes/*.md` · `docs/context/AWS-DEPLOY.md` · `src/db/repositories/README.md` · `src/db/schema.ts` · `src/db/migrations/` · `src/auth/guards.ts` · `proxy.ts` · `src/data/checklist.ts` · `src/data/compliance.ts` · `package.json` · `docker-compose.yml` · `tsconfig.json` · `.env.example` · the full `app/` route tree · the `src/` tree · and `git status` / `git log` on branch `client-dashboard-v7`.

Where this document and the repository disagree, **the repository wins** — and `docs/context/STATE.md` and `HANDOFF.md` are the live record.

End of knowledge transfer.